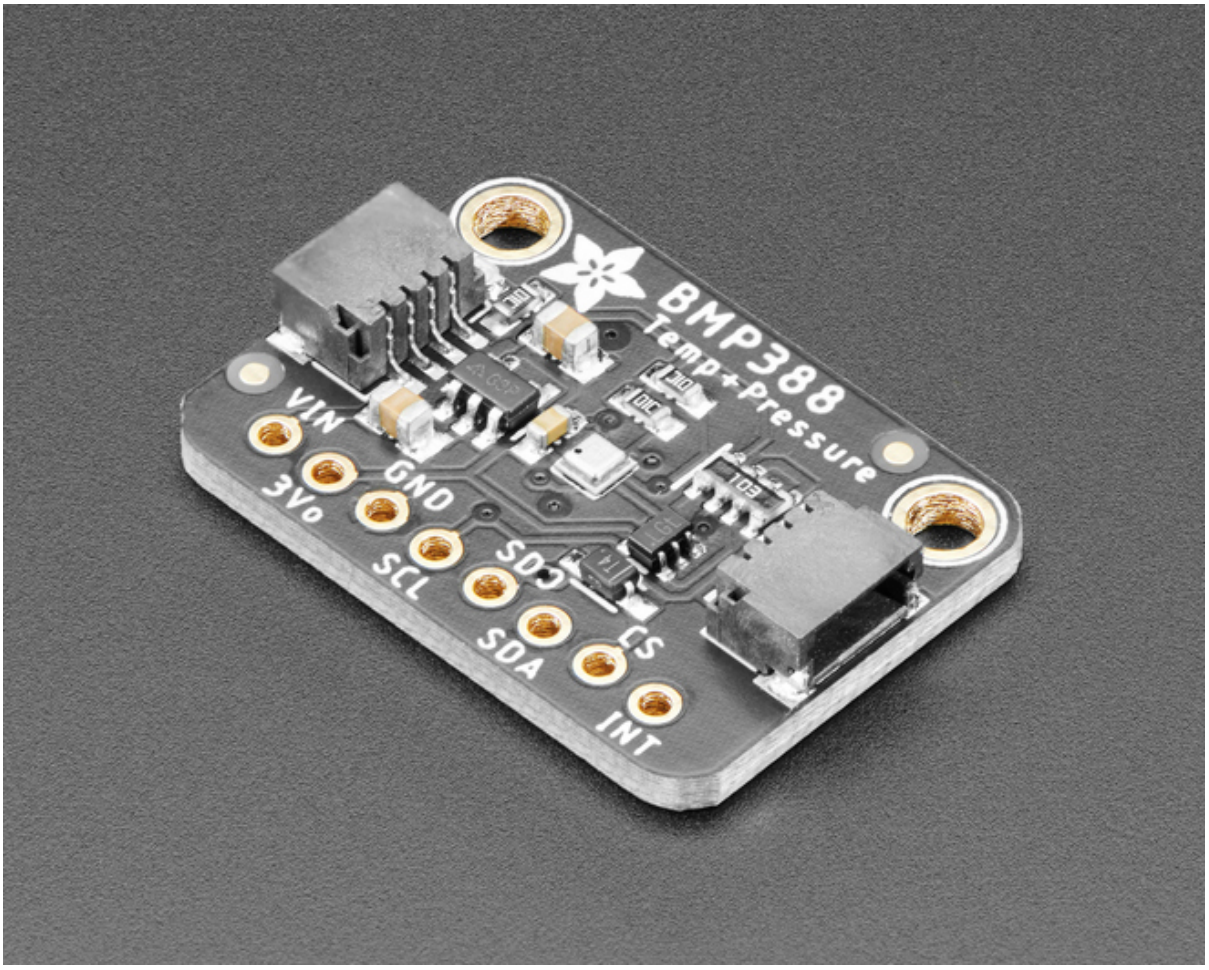
Adafruit BMP388 and BMP390 - Precision Barometric Pressure and Altimeter

Created by Kattni Rembor



https://learn.adafruit.com/adafruit-bmp388-bmp390-bmp3xx

Last updated on 2026-08-19 04:17:40 PM UTC

Table of Contents

Overview 3

Pinouts 7

- Power Pins
- SPI Logic pins
- I2C Logic pins
- Address Jumper

Assembly 9

- Prepare the header strip:
- Add the breakout board:
- And Solder!

Arduino 13

- I2C Wiring
- SPI Wiring
- Download Adafruit_BMP3XX library
- Load Demo
- Example Code

Arduino Docs 21

Python & CircuitPython 21

- CircuitPython Microcontroller Wiring
- Python Computer Wiring
- CircuitPython Installation of BMP3XX Library
- Python Installation of BMP3XX Library
- CircuitPython & Python Usage
- Full Example Code

Python Docs 30

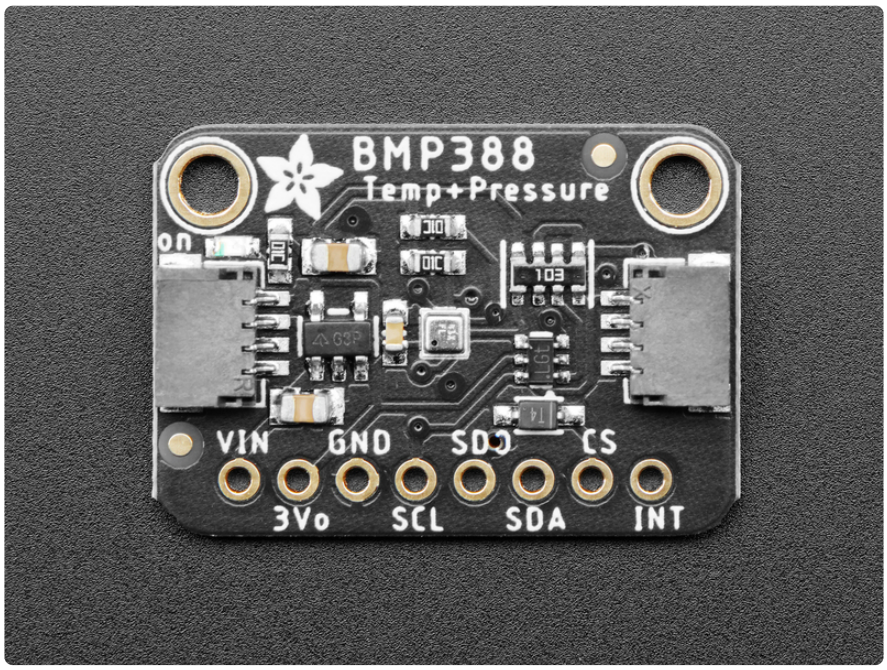
WipperSnapper 31

- What is WipperSnapper
- Wiring
- Usage

Downloads 39

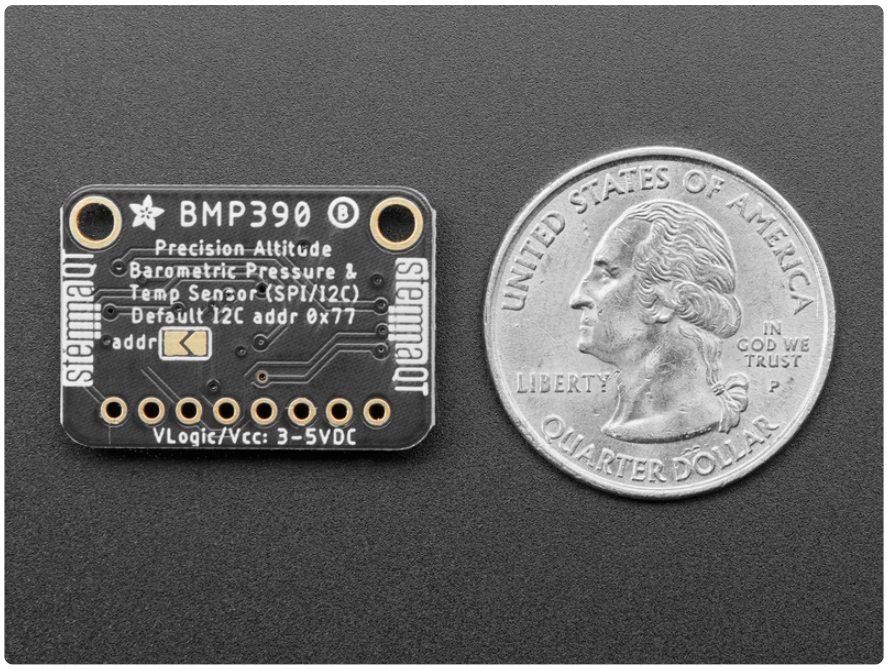
- Files
- Schematic and Fab Print for BMP390
- Schematic and Fab Print for BMP388 STEMMA QT Version
- Schematic and Fab Print for BMP388 Original Version
- 3D Model of BMP390
- 3D Model of BMP388 QT

Overview



Bosch has been a leader in barometric pressure sensors, from the BMP085 (http://adafru.it/391). BMP180 (http://adafru.it/1603), and BMP280 (http://adafru.it/2651)... now we've got the next generation, the **Adafruit BMP388 and BMP390 Precision Barometric Pressure sensors**. As you would expect, this sensor is similar to its earlier versions but even better. The BMP3xx has better precision than the BMP2xx series which makes it excellent for environmental sensing or as a **precision altimeter**. It can even be used in either I2C and SPI configurations.

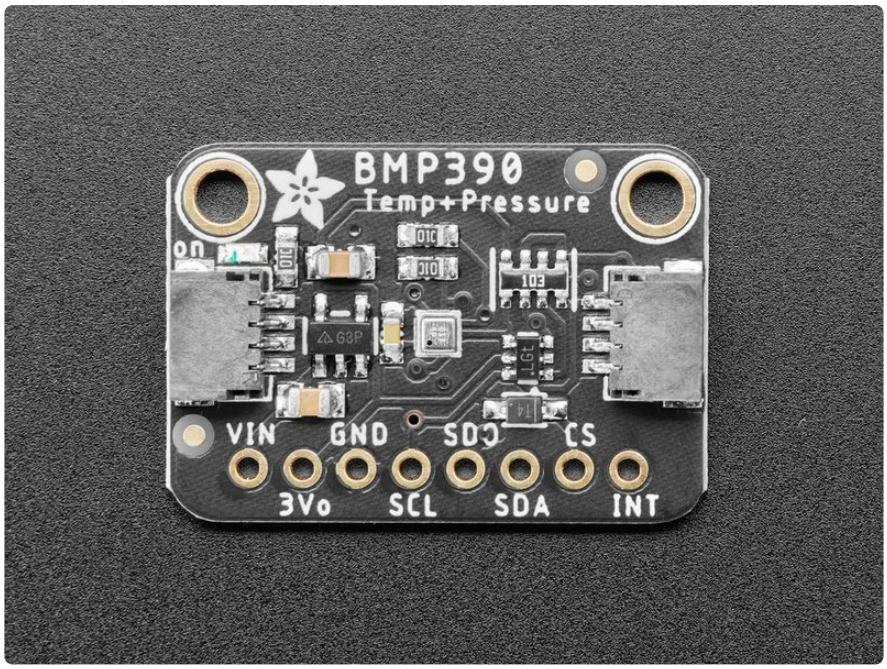
The BMP3xx is the next-generation of sensors from Bosch, and is the upgrade to the BMP280 - with a low altitude noise as low as 0.1m and the same fast conversion time. And like the previous BMP280, you can use I2C or SPI. For simple easy wiring, go with I2C. If you want to connect a bunch of sensors without worrying about I2C address collisions, go with SPI.



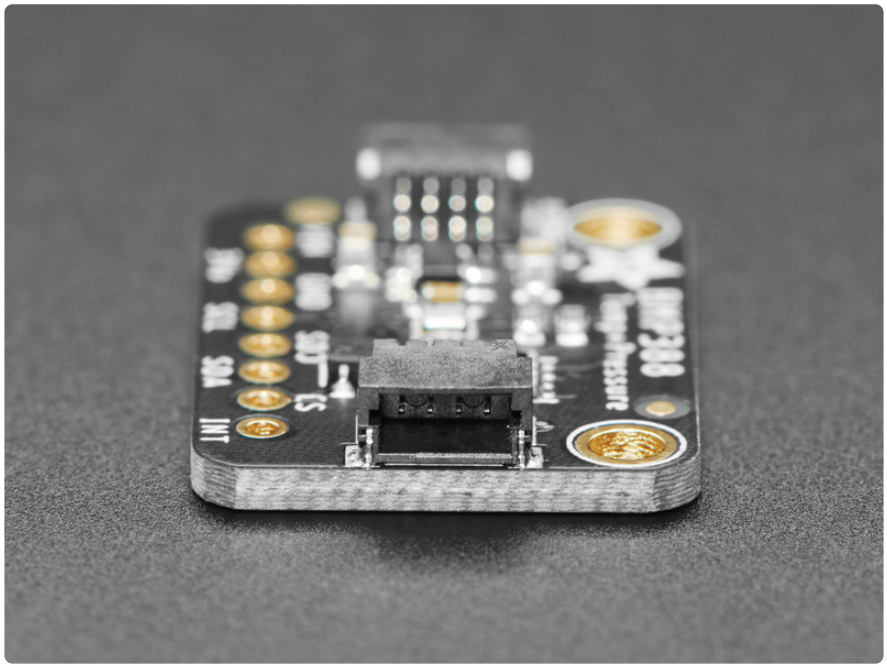
- This **BMP388** sensor has a relative accuracy of 8 Pascals, which translates to about ± 0.5 meter of altitude
- This **BMP390** sensor has a relative accuracy of 3 Pascals, which translates to about ± 0.25 meter of altitude

Compare to the BMP280's 12 Pascal/ ±1 meter!

The datasheet sort of implies they intend this sensor to be used for drones and quadcopters, to keep altitude stable, but you could also use this for wearables or any project that wants to track height-above-sea-level. Note that for absolute height you'll still need to enter in the barometric pressure at sea level, if the weather changes, but that's true of every altimeter sensor that uses pressure. You can also measure temperature with ±0.5°C accuracy.



Nice sensor right? So we made it easy for you to get right into your next project. The surface-mount sensor is soldered onto a PCB and comes with a 3.3V regulator and level shifting so you can use it with a 3V or 5V logic microcontroller without worry.

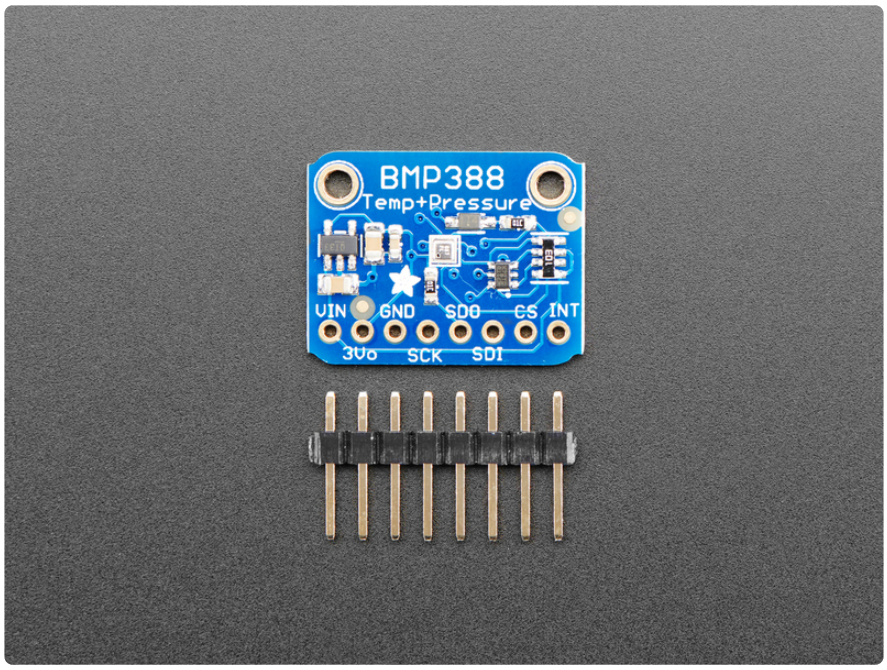


We also made these boards in the **STEMMA QT** form factor (https://adafru.it/LBQ), making them easy to interface with. The STEMMA QT connectors (https://adafru.it/JqB) on either side are compatible with the SparkFun Qwiic (https://adafru.it/Fpw) I2C connectors. This allows you to make solderless connections between your

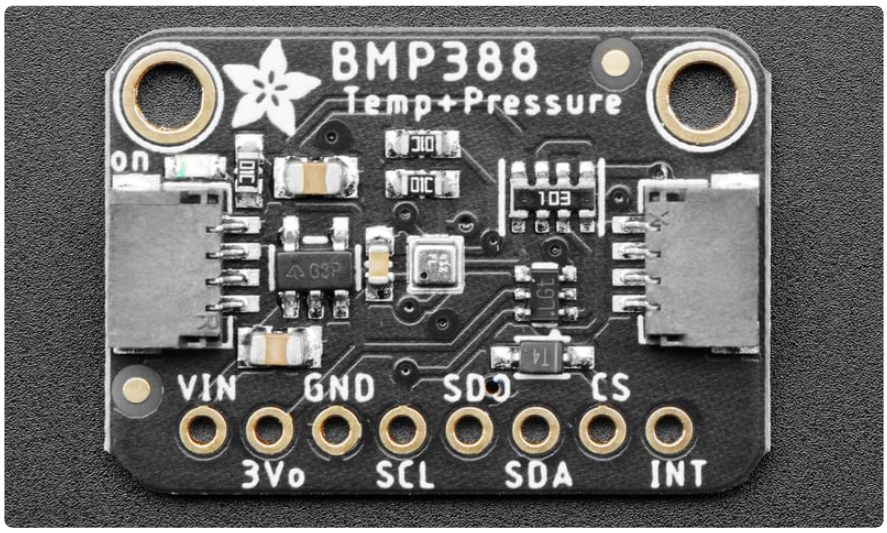
development board and the BMP390 or to chain it with a wide range of other sensors and accessories using a **compatible cable** (https://adafru.it/JnB).

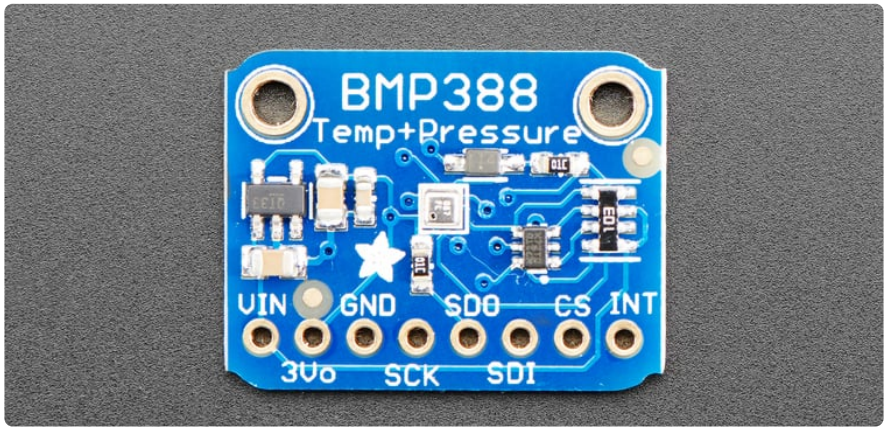
Text emphasized with a blue exclamation:

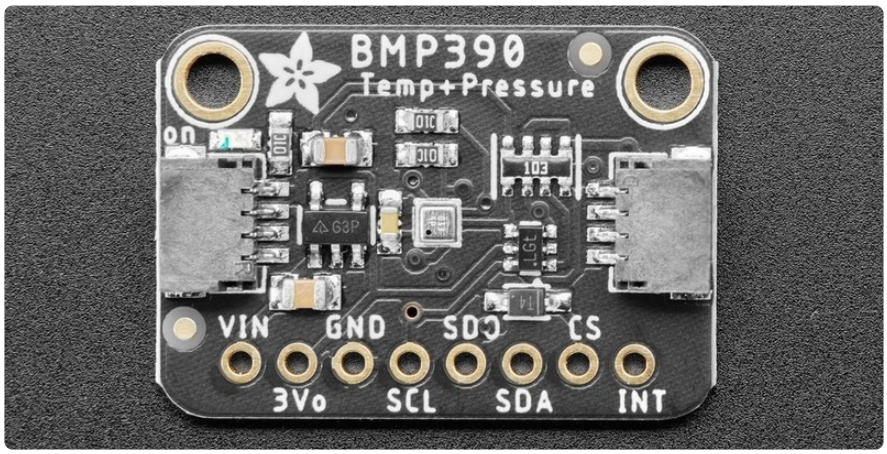
There are two versions of this board - the STEMMA QT version shown above, and the original header-only version shown below. Code works the same on both!



Pinouts


BMP388
Temp+Pressure
VIN
GND
SDO
CS
3Vo
SCL
SDA
INT


BMP388
Temp+Pressure
VIN
GND
SDO
CS
INT
3Vo
SCK
SDI


BMP390
Temp+Pressure
VIN
GND
SDO
CS
3Vo
SCL
SDA
INT

Text emphasized with a blue exclamation:

The BMP388 and BMP390 have the same exact pinout even though the names are a little different!

Power Pins

- **Vin** - this is the power pin. Since the sensor chip uses 3 VDC, we have included a voltage regulator on board that will take 3-5VDC and safely convert it down. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V
- **3Vo** - this is the 3.3V output from the voltage regulator, you can grab up to 100mA from this if you like
- **GND** - common ground for power and logic

SPI Logic pins

All pins going into the breakout have level shifting circuitry to make them 3-5V logic level safe. Use whatever logic level is on **Vin!**

- **SCK** - This is the **S**PI **C**lo**k** pin, its an input to the chip and can use 3 - 5V logic.
- **SDO** - this is the **S**erial **D**ata **O**ut / **M**icrocontroller **I**n **S**ensor **O**ut pin, for data sent from the BMP3xx to your processor. Logic level is 3.3V output, so can be read by 5V microcontrollers.
- **SDA** - this is the **S**erial **Da**ta In / **M**icrocontroller **O**ut **S**ensor **I**n pin, for data sent from your processor to the BMP3xx. Its an input to the chip and can use 3 - 5V logic. (SDI on original version)
- **CS** - this is the **C**hip **S**elect pin, drop it low to start an SPI transaction. Its an input to the chip and can use 3 - 5V logic.
- **INT** - this is the **Int**errupt pin. The BMP3xx can send an output signal to tell you when data is read (we don't use this in our libraries but it is available for your use). The logic level is 3.3V output, so it can be read by 5V microcontrollers.

If you want to connect multiple BMP3xx sensors to one microcontroller, have them share the SDI, SDO and SCK pins. Then assign each one a unique CS pin.

I2C Logic pins

- **SCK** - this is also the I2C clock pin, connect to your microcontroller's I2C clock line.
- **SDA** - this is also the I2C data pin, connect to your microcontroller's I2C data line. (SDI on original version)
- STEMMA QT **(https://adafru.it/Ft4) -** These connectors allow you to connectors to dev boards with **STEMMA QT** connectors or to other things with various associated accessories (https://adafru.it/Ft6)

Leave the **CS** and **SDO** pins disconnected for I2C use.

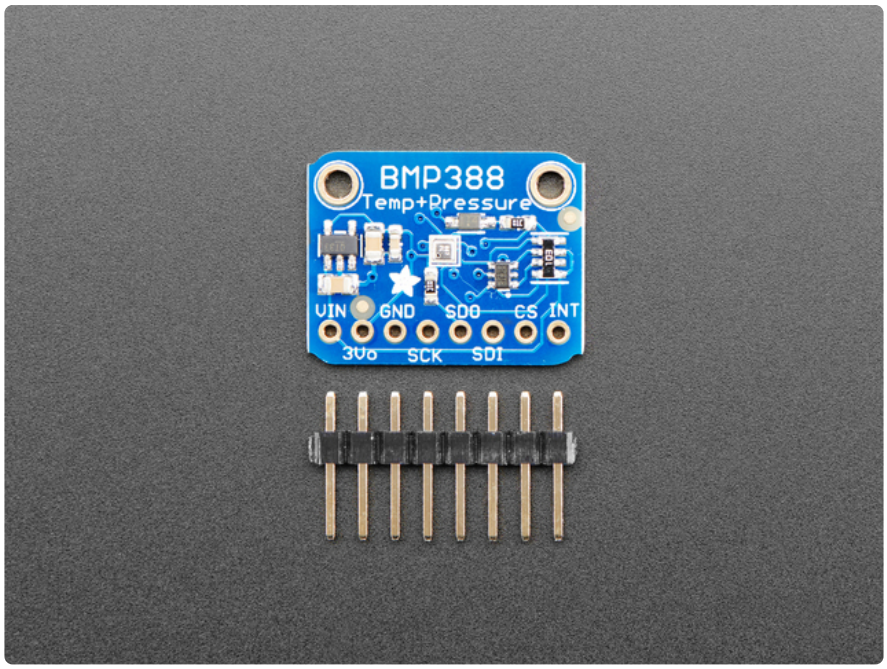
Address Jumper

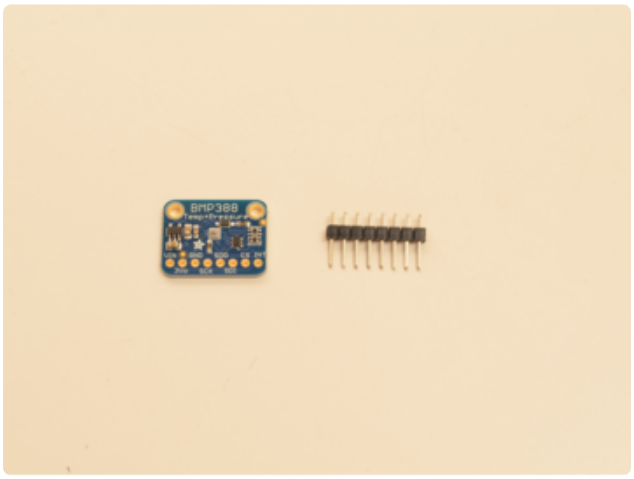
- **ADDR** - This is the I2C address selection jumper. The default I2C address - when this jumper is disconnected - is **0x77**. On the STEMMA QT Versions you can change the I2C address by shorting this jumper. Once shorted the I2C address is **0x76**.

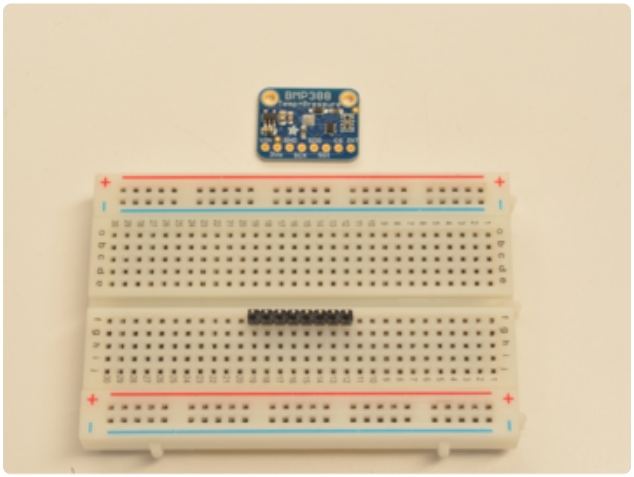
Text emphasized with a blue exclamation:	Only the versions of these boards with STEMMA QT connectors have the ADDR jumper

Assembly

Text emphasized with a blue exclamation:	The BMP388 and BMP390 have the same soldering technique even though the names and shapes are a little different!

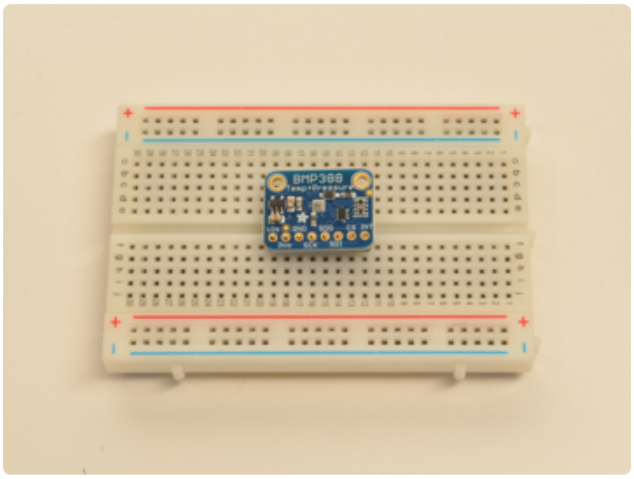






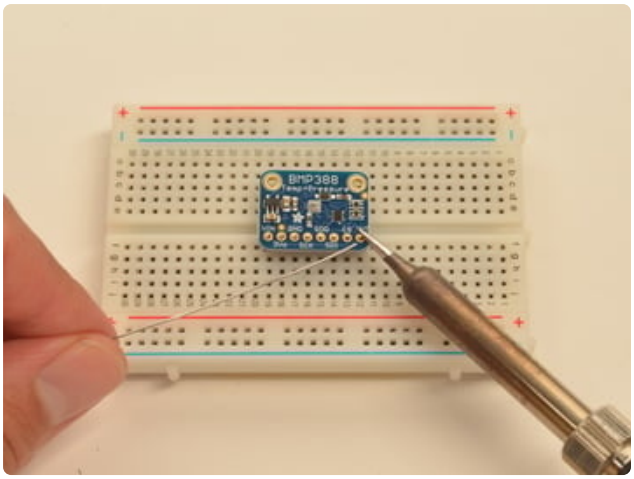
Prepare the header strip:

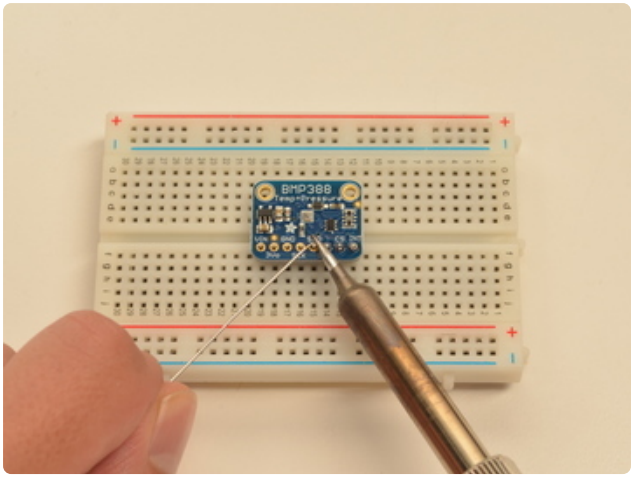
Cut the strip to length if necessary. It will be easier to solder if you insert it into a breadboard - **long pins down**

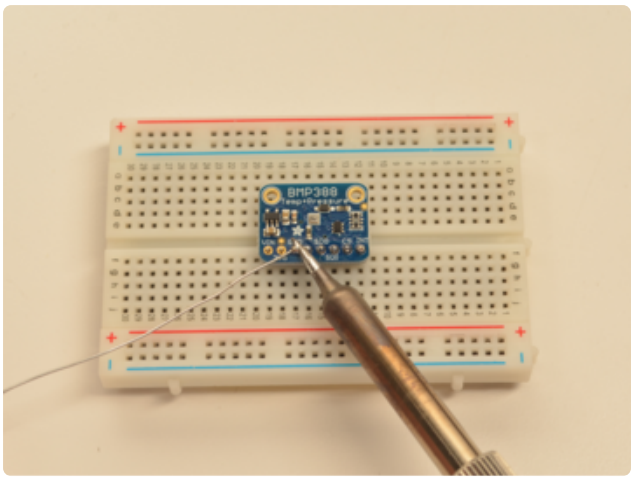


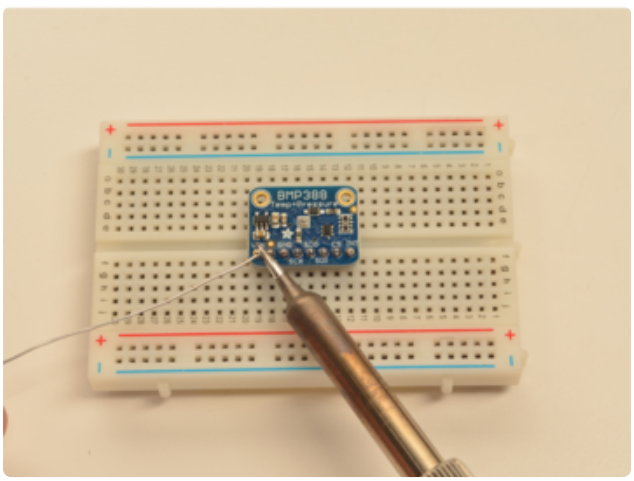
Add the breakout board:

Place the breakout board over the pins so that the short pins poke through the breakout pads





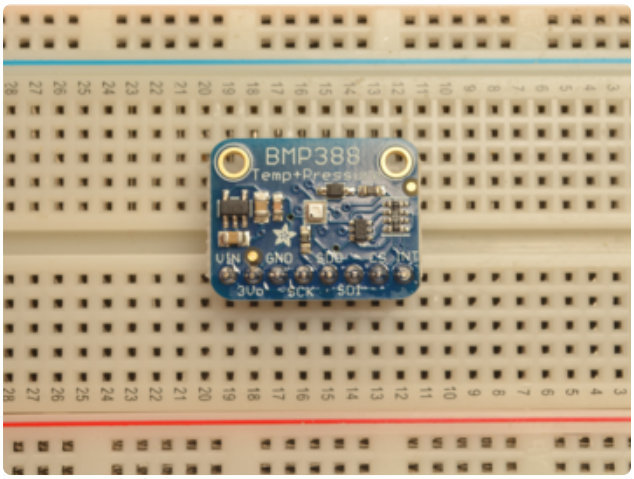




And Solder!

Be sure to solder all 8 pins for reliable electrical contact.

(For tips on soldering, be sure to check out our Guide to Excellent Soldering (https://adafru.it/aTk)).



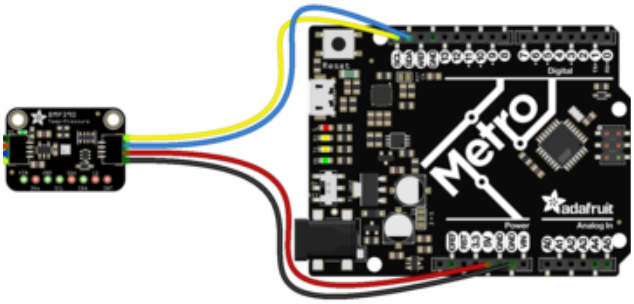
You're done! Check your solder joints visually and continue onto the next steps.

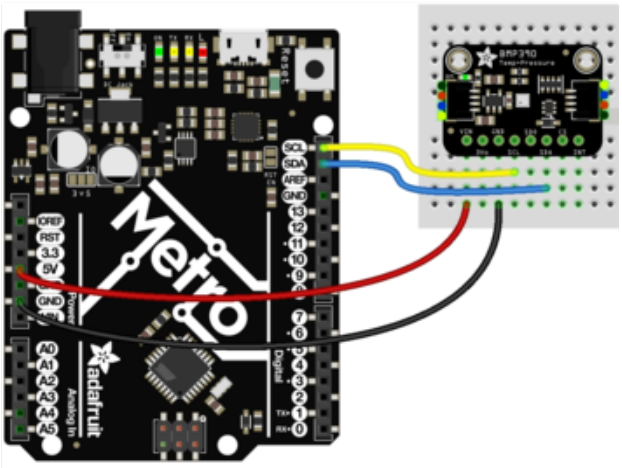
Arduino

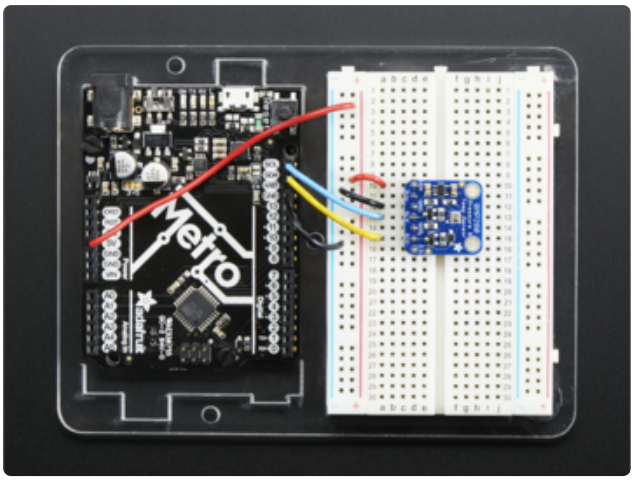
You can easily wire this breakout to any microcontroller, we'll be using an Arduino Uno/328P compatible. For another kind of microcontroller, as long as you have 4 available pins it is possible to 'bit-bang SPI' or you can use two I2C pins, but usually those pins are fixed in hardware. Just check out the library, then port the code.

I2C Wiring

Use this wiring if you want to connect via I2C interface:





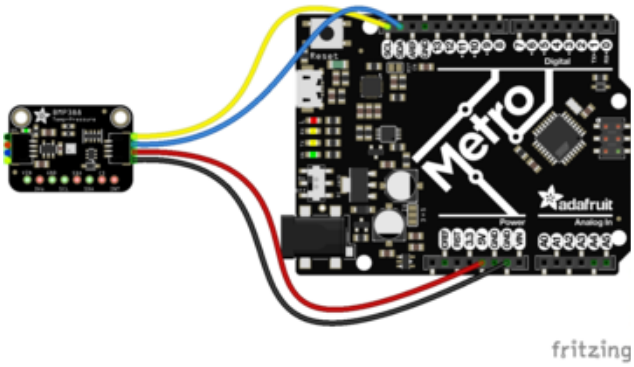


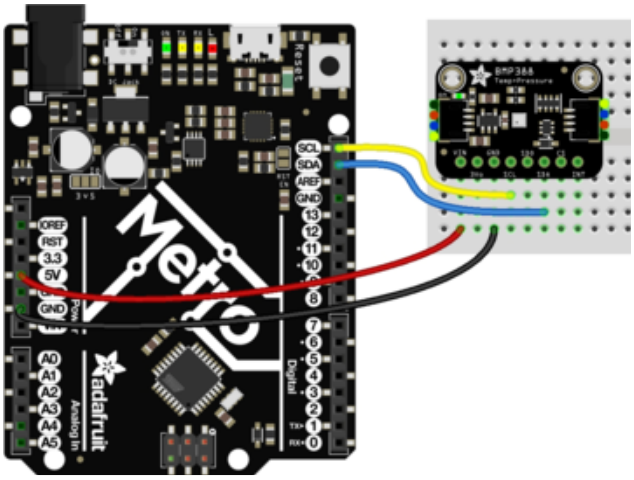
Connect **Vin** to the power supply, 3-5V is fine. **(Red wire on STEMMA QT version.)** Use the same voltage that the microcontroller logic is based off of. For most older Arduinos, that is 5V.

Connect **GND** to common power/data ground. **(Black wire on STEMMA QT version.)**

Connect the **SCK/SCL** pin to the I2C clock **SCL** pin on your Arduino. **(Yellow wire on STEMMA QT version.)** On an UNO & '328 based Arduino, this is also known as **A5**, on a Mega it is also known as **digital 21** and on a Leonardo/Micro, **digital 3.**

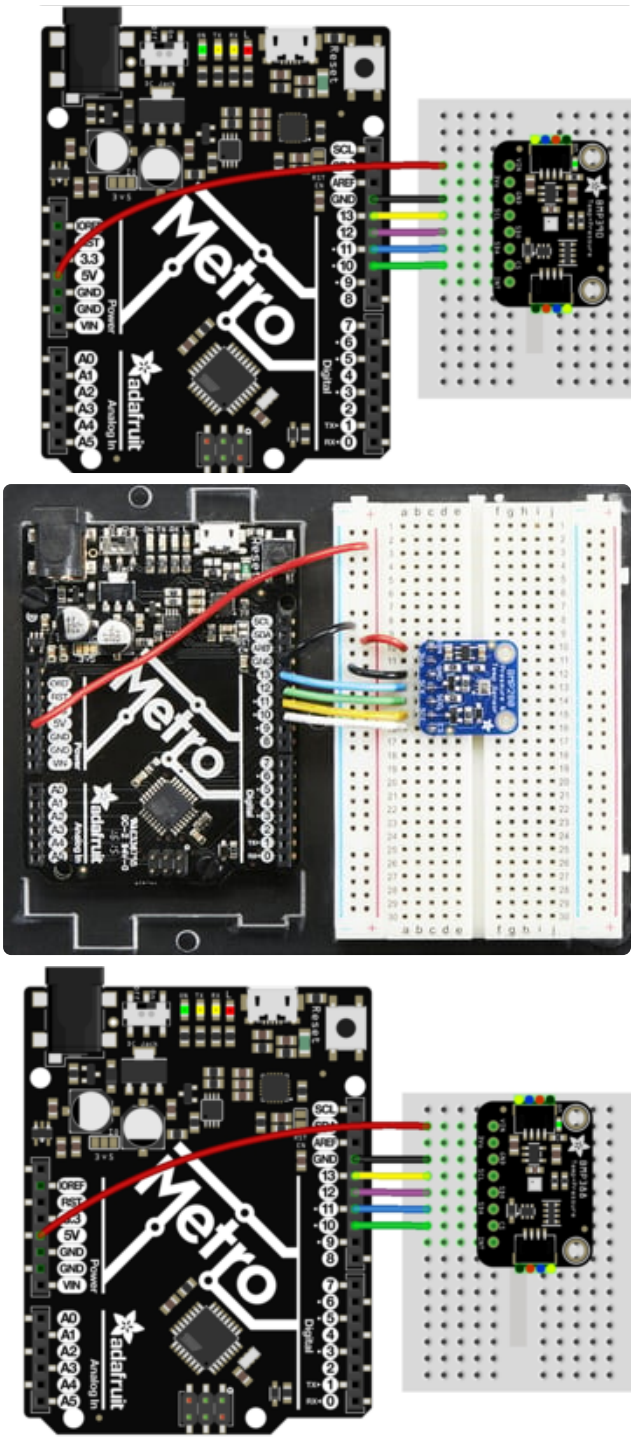
Connect the **SDI/SDA** pin to the I2C data **SDA** pin on your Arduino. **(Blue wire on STEMMA QT version.)** On an UNO & '328 based Arduino, this is also known as **A4**, on a Mega it is also known as **digital 20** and on a Leonardo/Micro, **digital 2.**





SPI Wiring

Since this is a SPI-capable sensor, we can use hardware or 'software' SPI. To make wiring identical on all Arduinos, we'll begin with 'software' SPI. The following pins should be used:



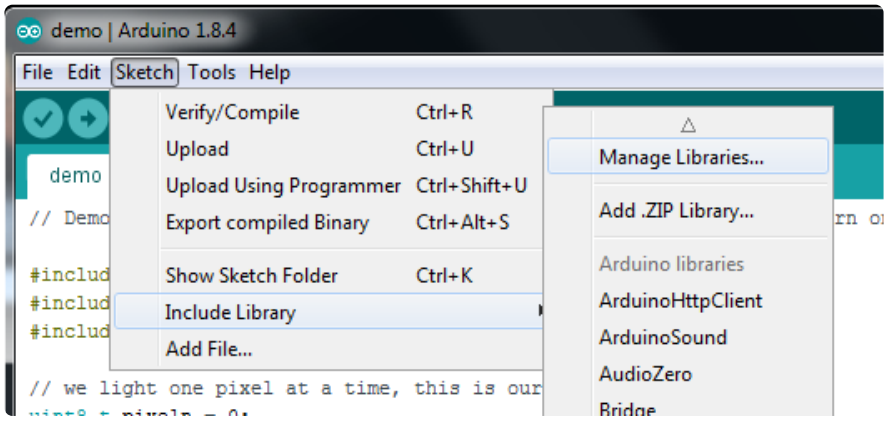
Connect **Vin** to the power supply, 3V or 5V is fine. Use the same voltage that the microcontroller logic is based off of. For most Arduinos, that is 5V.
Connect **GND** to common power/data ground.
Connect the **SCK/SCL** pin to **Digital #13** but any pin can be used later.
Connect the **SDO** pin to **Digital #12** but any pin can be used later.
Connect the **SDI/SDA** pin to **Digital #11** but any pin can be used later.
Connect the **CS** pin **Digital #10** but any pin can be used later.

Later on, once we get it working, we can adjust the library to use hardware SPI if you desire, or change the pins assignments.

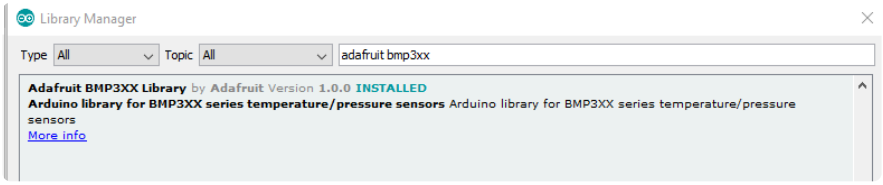
Download Adafruit_BMP3XX library

To begin reading sensor data, you will need to [install the Adafruit_BMP3XX library (code on our github repository)](https://adafru.it/EdV). It is available from the Arduino library manager so we recommend using that.

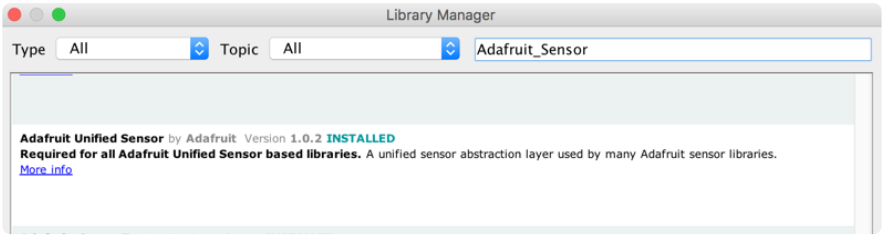
From the IDE open up the library manager...



And type in **adafruit bmp3xx** to locate the library. Click **Install**



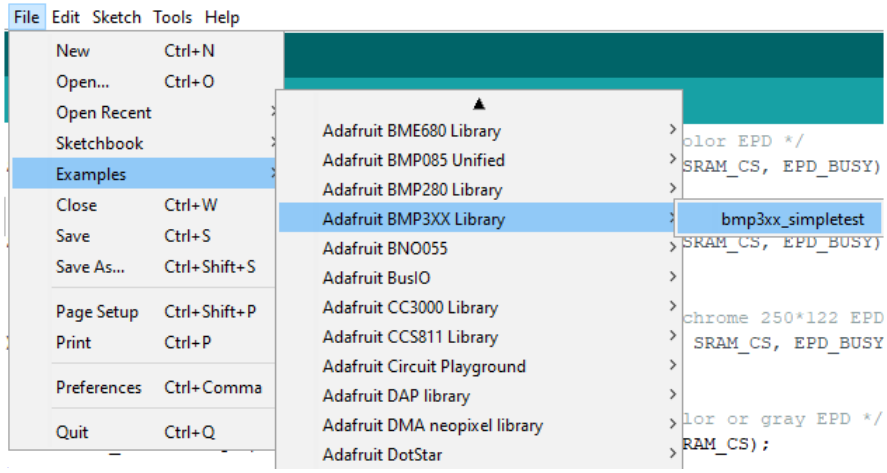
You'll also need to install the **Adafruit Unified Sensor** library



We also have a great tutorial on Arduino library installation at: http://learn.adafruit.com/adafruit-all-about-arduino-libraries-install-use (https://adafru.it/aYM)

Load Demo

Open up **File -> Examples -> Adafruit_BMP3XX -> bmp3xx_simpletest** and upload to your Arduino wired up to the sensor.



Depending on whether you are using I2C or SPI, change the pin names and comment or uncomment the following lines.

```
#define BMP_SCK 13
#define BMP_MISO 12
#define BMP_MOSI 11
#define BMP_CS 10

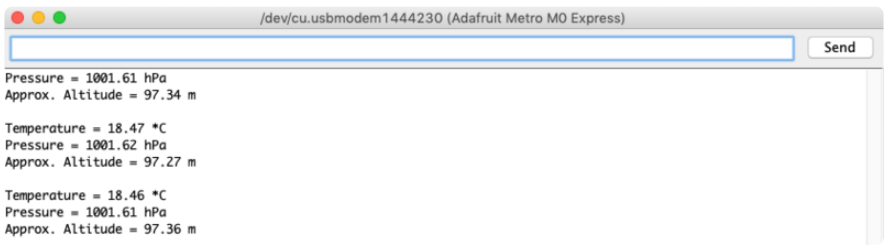
#define SEALEVELPRESSURE_HPA (1013.25)

Adafruit_BMP3XX bmp; // I2C
//Adafruit_BMP3XX bmp(BMP_CS); // hardware SPI
//Adafruit_BMP3XX bmp(BMP_CS, BMP_MOSI, BMP_MISO, BMP_SCK);  // Software SPI
```

Once uploaded to your Arduino, open up the serial console at 115200 baud to see data being printed out:

Text emphasized with a yellow arrow:

The first reading may be inaccurate. You should look to read the sensor once and discard the reading. Subsequent readings should be accurate.



Temperature is calculated in degrees C, you can convert this to F by using the classic F = C * 9/5 + 32 equation.

Pressure is returned in the SI units of **Pascals**. 100 Pascals = 1 hPa = 1 millibar. Often times barometric pressure is reported in millibar or inches-mercury (Hg). For future reference 1 pascal =0.000295333727 inches of mercury, or 1 inch Hg = 3386.39 Pascal. So if you take the Pascal value of say 100734 and divide by 3389.39 you'll get 29.72 inches-Hg.

You can also calculate Altitude. **However, you can only really do a good accurate job of calculating altitude if you know the hPa pressure at sea level for your location and day!** The sensor is quite precise but if you do not have the data updated for the current day, then it can be difficult to get more accurate than 10 meters.

Pass in the current sea level pressure in **hPa** - so the value will be somewhere around ~1000. You can also test with the generic 1013.25 value.

Example Code

The following example code is part of the standard library. It illustrates how you can retrieve sensor data from the BMP388 for the temperature, pressure and approximate altitude:

```
/***************************************************************************
  This is a library for the BMP3XX temperature & pressure sensor

  Designed specifically to work with the Adafruit BMP388 Breakout
  ----> http://www.adafruit.com/products/3966

  These sensors use I2C or SPI to communicate, 2 or 4 pins are required
  to interface.

  Adafruit invests time and resources providing this open source code,
  please support Adafruit and open-source hardware by purchasing products
  from Adafruit!

  Written by Limor Fried & Kevin Townsend for Adafruit Industries.
  BSD license, all text above must be included in any redistribution
 ***************************************************************************/

#include <Wire.h>
#include <SPI.h>
#include <Adafruit_Sensor.h>
#include "Adafruit_BMP3XX.h"

#define BMP_SCK 13
#define BMP_MISO 12
#define BMP_MOSI 11
#define BMP_CS 10
```

```
27
28  #define SEALEVELPRESSURE_HPA (1013.25)
29
30  Adafruit_BMP3XX bmp;
31
32  void setup() {
33    Serial.begin(115200);
34    while (!Serial);
35    Serial.println("Adafruit BMP388 / BMP390 test");
36
37    if (!bmp.begin_I2C()) {   // hardware I2C mode, can pass in address & alt Wire
38    //if (! bmp.begin_SPI(BMP_CS)) {  // hardware SPI mode
39    //if (! bmp.begin_SPI(BMP_CS, BMP_SCK, BMP_MISO, BMP_MOSI)) {  // software SPI mode
40      Serial.println("Could not find a valid BMP3 sensor, check wiring!");
41      while (1);
42    }
43
44    // Set up oversampling and filter initialization
45    bmp.setTemperatureOversampling(BMP3_OVERSAMPLING_8X);
46    bmp.setPressureOversampling(BMP3_OVERSAMPLING_4X);
47    bmp.setIIRFilterCoeff(BMP3_IIR_FILTER_COEFF_3);
48    bmp.setOutputDataRate(BMP3_ODR_50_HZ);
49  }
50
51  void loop() {
52    if (! bmp.performReading()) {
53      Serial.println("Failed to perform reading :(");
54      return;
55    }
56    Serial.print("Temperature = ");
57    Serial.print(bmp.temperature);
58    Serial.println(" *C");
59
60    Serial.print("Pressure = ");
61    Serial.print(bmp.pressure / 100.0);
62    Serial.println(" hPa");
63
64    Serial.print("Approx. Altitude = ");
65    Serial.print(bmp.readAltitude(SEALEVELPRESSURE_HPA));
66    Serial.println(" m");
67
68    Serial.println();
69    delay(2000);
70  }
```

https://github.com/adafruit/Adafruit_BMP3XX/blob/master/examples/bmp3xx_simpletest/bmp3xx_simpletest.ino

Text emphasized with a purple info:

The first value read may not be accurate as the sensor gets going. If you find this behavior, make one sensor read and ignore the data. Subsequent values should then be ok.

Arduino Docs

Arduino Docs (https://adafru.it/EqL)

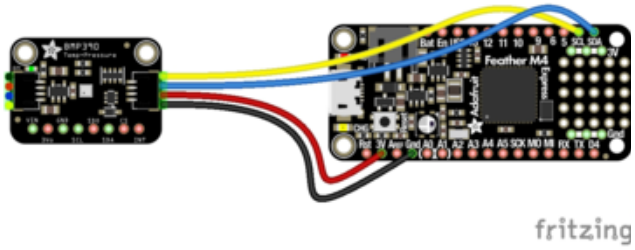
Python & CircuitPython

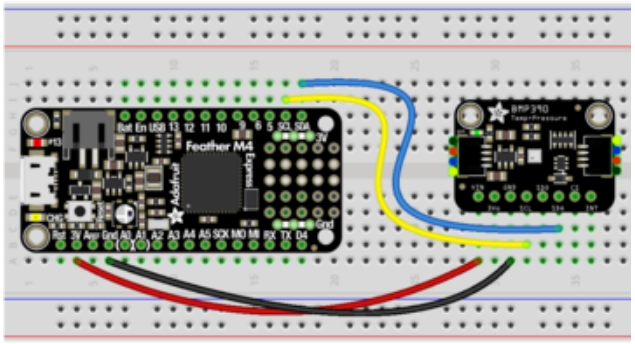
It's easy to use the BMP388 sensor with CircuitPython and the Adafruit CircuitPython BMP3XX (https://adafru.it/Ei6) module. This module allows you to easily write Python code that reads the barometric pressure, temperature and more from the sensor.

You can use this sensor with any CircuitPython microcontroller board or with a computer that has GPIO and Python thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library (https://adafru.it/BSN).

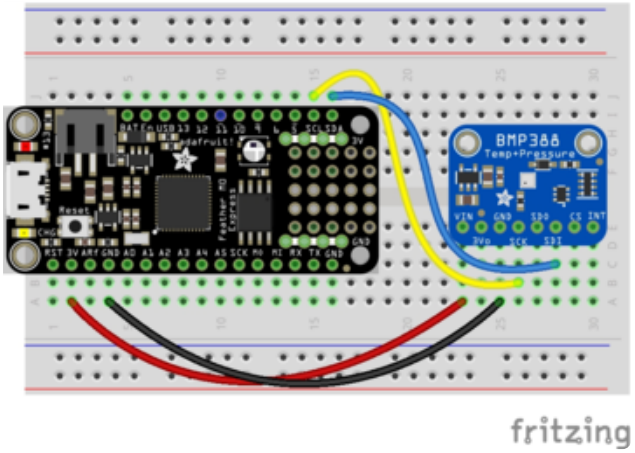
CircuitPython Microcontroller Wiring

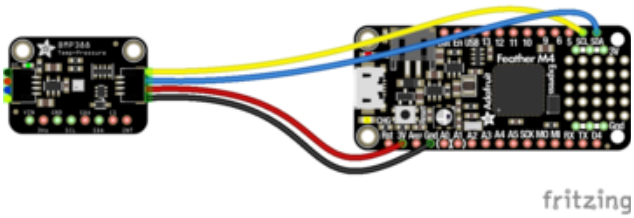
First wire up a BMP3xx to your board as shown below. You can use either I2C or SPI wiring, although it's recommended to use I2C for simplicity. Here's an example of wiring a Feather to the sensor with I2C:

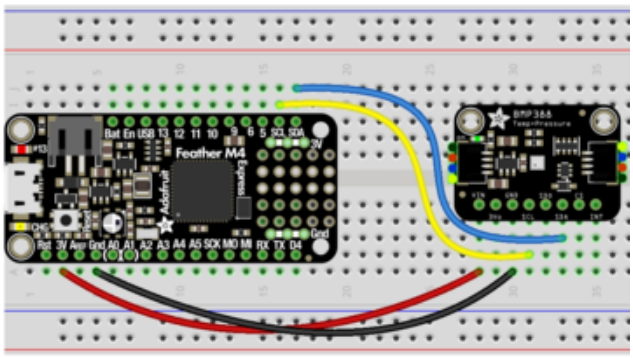




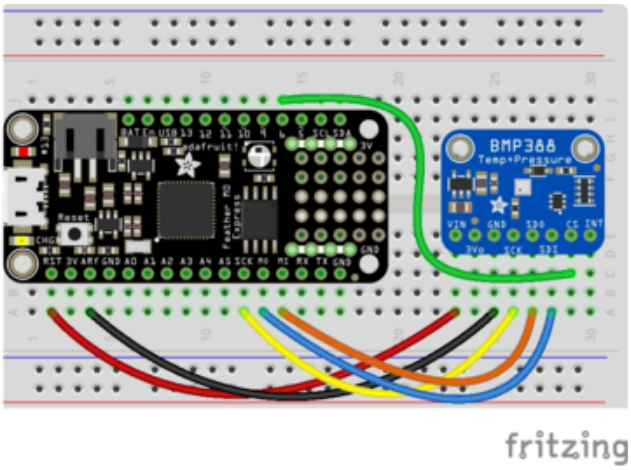
Board 3V to sensor VIN (red wire on STEMMA QT version)
Board GND to sensor GND (black wire on STEMMA QT version)
Board SCL to sensor SCK/SCL (yellow wire on STEMMA QT version)
Board SDA to sensor SDI/SDA (blue wire on STEMMA QT version)

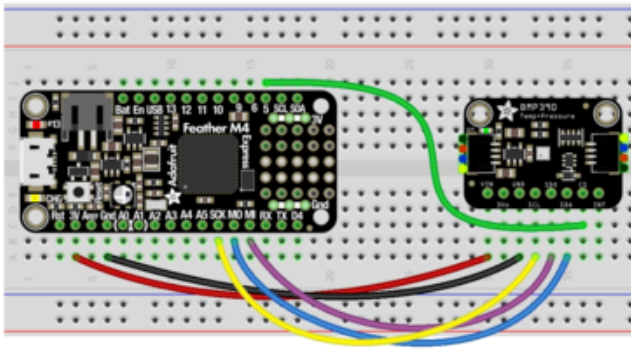




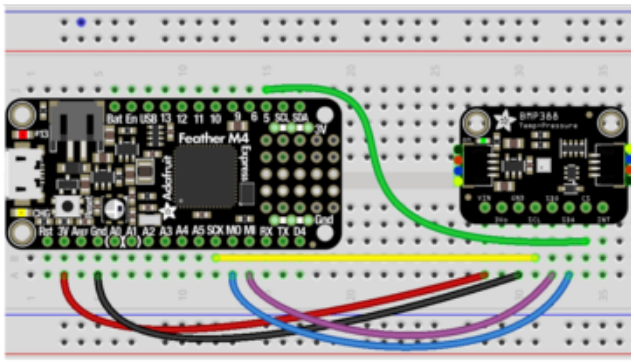


And an example of a Feather wired with hardware SPI:





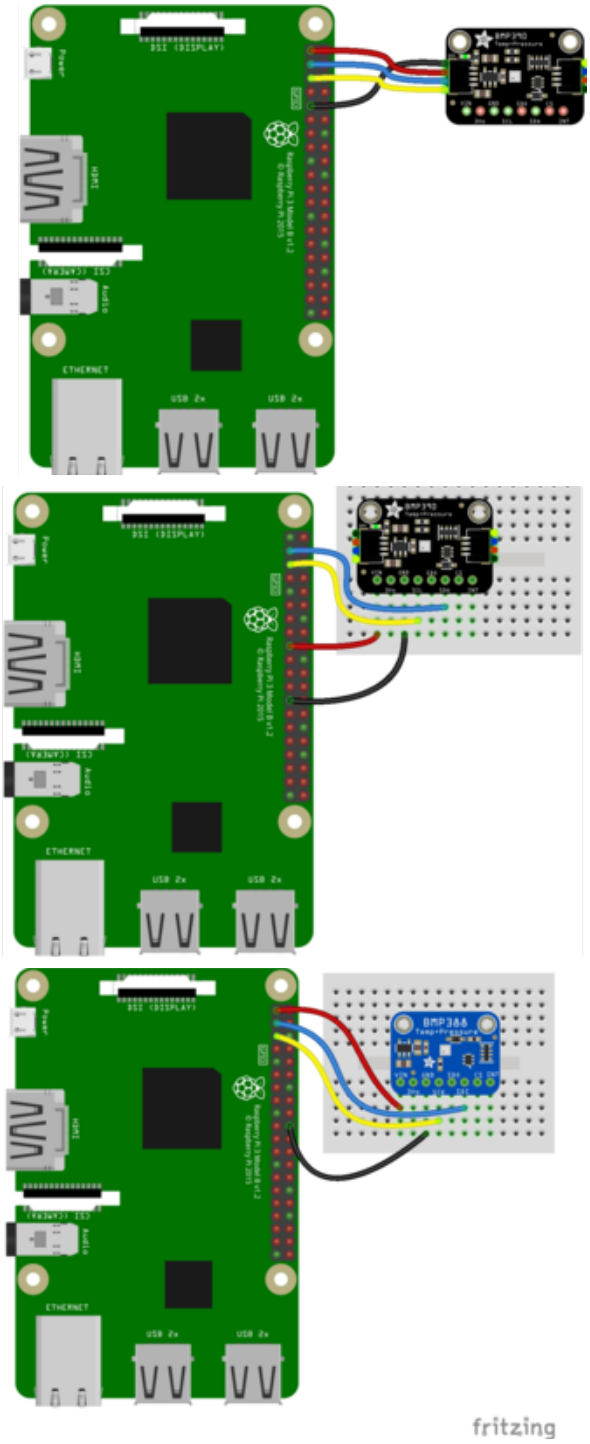
Board 3V to **sensor VIN**
Board GND to **sensor GND**
Board SCK to **sensor SCK/SCL**
Board MOSI to **sensor SDI/SDA**
Board MISO to **sensor SDO**
Board D5 to **sensor CS** (or use any other free digital I/O pin)



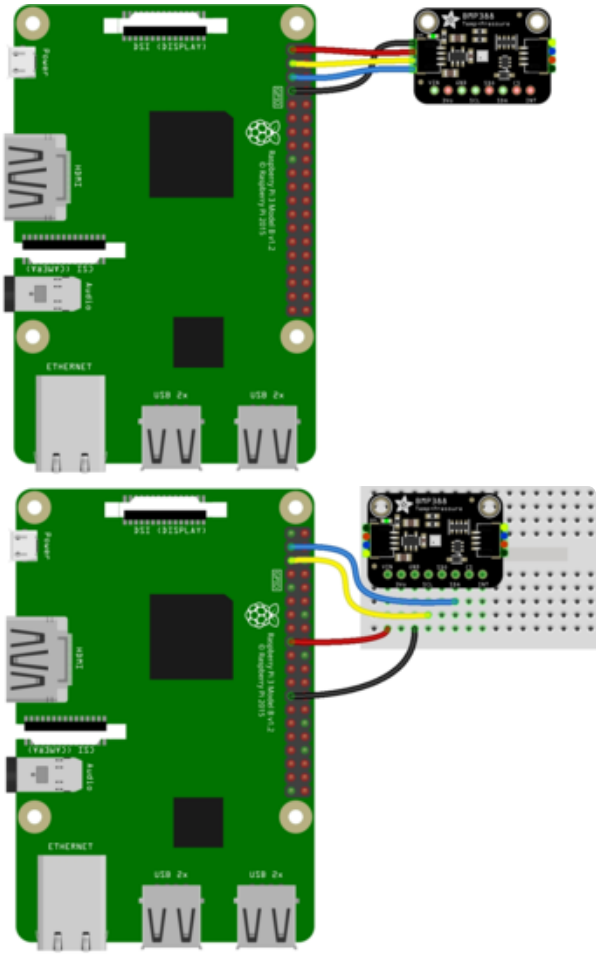
Python Computer Wiring

Since there's dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafru.it/BSN) (https://adafru.it/BSN).

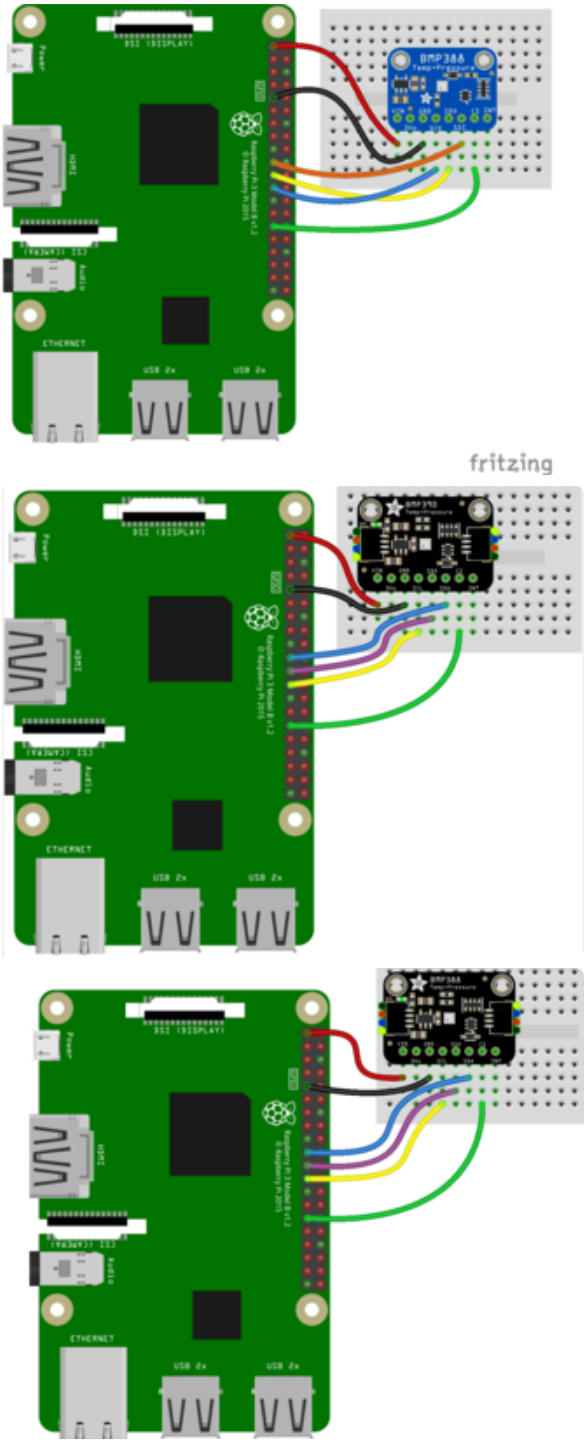
Here's the Raspberry Pi wired with I2C:



Pi 3V3 to **sensor VIN**
Pi GND to **sensor GND**
Pi SCL to **sensor SCK/SCL**
Pi SDA to **sensor SDI/SDA**



And an example on the Raspberry Pi 3 Model B wired with SPI:



Pi 3V3 to **sensor VIN**
Pi GND to **sensor GND**
Pi MOSI to **sensor SDI/SDA**
Pi MISO to **sensor SDO**
Pi SCLK to **sensor SCK/SCL**
Pi #5 to **sensor CS** (or use any other free GPIO pin)

CircuitPython Installation of BMP3XX Library

You'll need to install the Adafruit CircuitPython BMP3XX (https://adafru.it/Ei6) library on your CircuitPython board.

First make sure you are running the latest version of Adafruit CircuitPython (https://adafru.it/Amd) for your board.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from Adafruit's CircuitPython library

bundle (https://adafru.it/uap). Our CircuitPython starter guide has a great page on how to install the library bundle (https://adafru.it/ABU).

For non-express boards like the Trinket M0 or Gemma M0, you'll need to manually install the necessary libraries from the bundle:

- **adafruit_bmp3xx.mpy**
- **adafruit_bus_device**

Before continuing, make sure your board's **lib** folder or root filesystem has the **adafruit_bmp3xx.mpy,** and **adafruit_bus_device** files and folders copied over.

Next connect to the board's serial REPL (https://adafru.it/Awz)so you are at the CircuitPython >>> prompt.

Python Installation of BMP3XX Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready (https://adafru.it/BSN)!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-bmp3xx`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

CircuitPython & Python Usage

To demonstrate the usage of the sensor, we'll initialize it and read the pressure, temperature and more from the Python REPL.

If you're using an I2C connection, run the following code to import the necessary modules and initialize the I2C connection with the sensor:

```
import time
import board
import adafruit_bmp3xx
i2c = board.I2C()
bmp = adafruit_bmp3xx.BMP3XX_I2C(i2c)
```

Or if you're using a SPI connection run this code instead to setup the SPI connection and sensor:

```
import time
import board
import adafruit_bmp3xx
import digitalio
spi = board.SPI()
cs = digitalio.DigitalInOut(board.D5)
bmp = adafruit_bmp3xx.BMP3XX_SPI(spi, cs)
```

Now you're ready to read values from the sensor using any of these properties:

- `temperature` - The sensor temperature in degrees Celsius.
- `pressure` - The pressure in hPa.
- `altitude` - The altitude in meters.

For example to print temperature and pressure:

```
print("Pressure: {:6.1f}".format(bmp.pressure))
print("Temperature: {:5.2f}".format(bmp.temperature))
```

```
>>> print("Pressure: {:6.1f}".format(bmp.pressure))
Pressure:  985.0
>>> print("Temperature: {:5.2f}".format(bmp.temperature))
Temperature: 19.25
```

For altitude, you'll want to set the pressure at sea level for your location to get the most accurate measurement (remember these sensors can only infer altitude based on pressure and need a set calibration point). Look at your local weather report for a pressure at sea level reading and set the `sea_level_pressure` property:

```
bmp.sea_level_pressure = 1013.25
```

Then read the `altitude` property for a more accurate altitude reading (but remember this altitude will fluctuate based on atmospheric pressure changes!):

```
print('Altitude: {} meters'.format(bmp.altitude))
```

```
>>> bmp.sea_level_pressure = 1013.25
>>> print('Altitude: {} meters'.format(bmp.altitude))
Altitude: 238.16 meters
```



Text emphasized with a yellow arrow:

The first reading may be inaccurate. You should look to read the sensor once and discard the reading. Subsequent readings should be accurate.

That's all there is to using the BMP388 sensor with CircuitPython!

Full Example Code

```
# SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
# SPDX-License-Identifier: MIT

import time

import board

import adafruit_bmp3xx

# I2C setup
i2c = board.I2C()  # uses board.SCL and board.SDA
# i2c = board.STEMMA_I2C()  # For using the built-in STEMMA QT connector on a microcontroller
bmp = adafruit_bmp3xx.BMP3XX_I2C(i2c)

# SPI setup
# from digitalio import DigitalInOut, Direction
# spi = board.SPI()
# cs = DigitalInOut(board.D5)
# bmp = adafruit_bmp3xx.BMP3XX_SPI(spi, cs)

bmp.pressure_oversampling = 8
bmp.temperature_oversampling = 2

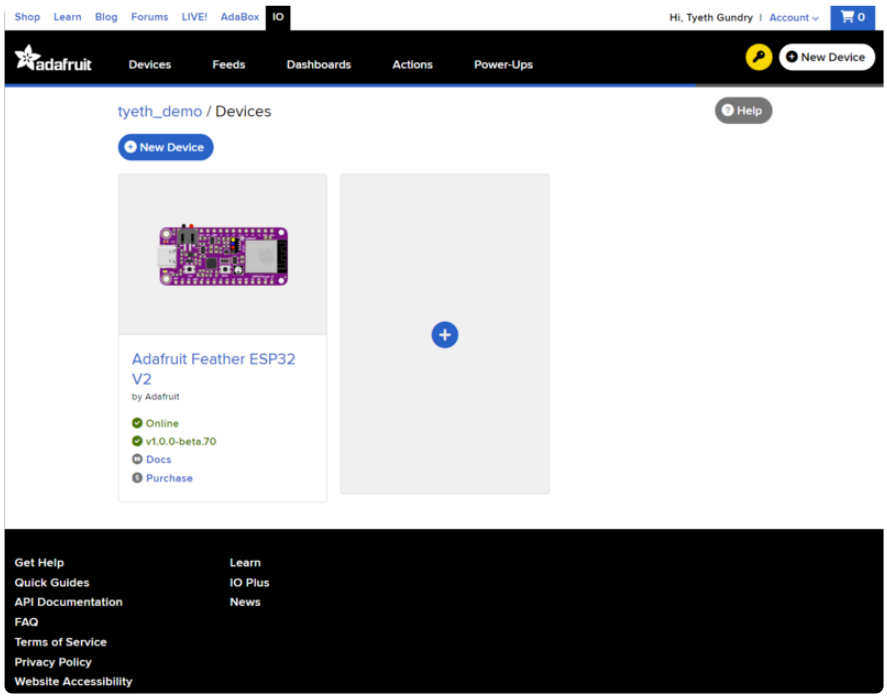
while True:
    print(f"Pressure: {bmp.pressure:6.4f}  Temperature: {bmp.temperature:5.2f}")
    time.sleep(1)
```

https://github.com/adafruit/Adafruit_CircuitPython_BMP3XX/blob/main/examples/bmp3xx_simpletest.py

Python Docs

Python Docs (https://adafru.it/Efl)

WipperSnapper



What is WipperSnapper

WipperSnapper is a firmware designed to turn any WiFi-capable board into an Internet-of-Things device without programming a single line of code. WipperSnapper connects to Adafruit IO (https://adafru.it/fsU), a web platform designed (by Adafruit! (https://adafru.it/Bo5)) to display, respond, and interact with your project's data.

Simply load the WipperSnapper firmware onto your board, add credentials, and plug it into power. Your board will automatically register itself with your Adafruit IO account.

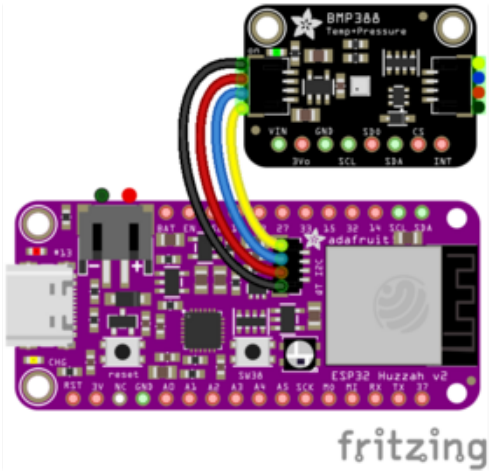
From there, you can add components to your board such as buttons, switches, potentiometers, sensors, and more! Components are dynamically added to hardware, so you can immediately start interacting, logging, and streaming the data your projects produce without writing code.

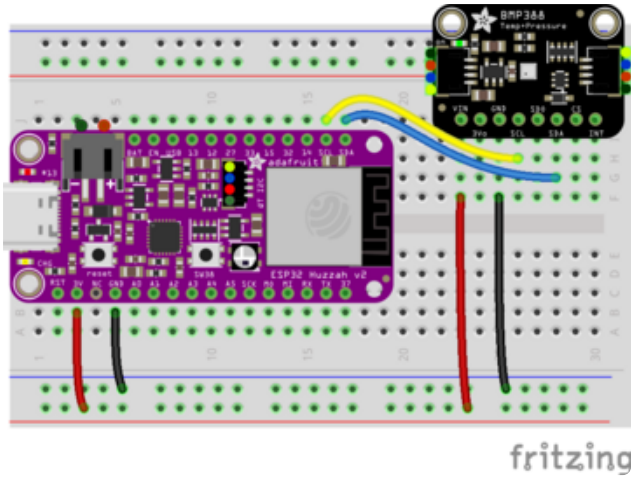
If you've never used WipperSnapper, click below to read through the quick start guide before continuing.

https://adafru.it/Vfd

Wiring

First, wire up a BMP388 to your board exactly as follows. Here is an example of the BMP388 wired to an Adafruit ESP32 Feather V2 (http://adafru.it/5400) using I2C with a STEMMA QT cable (no soldering required) (http://adafru.it/4210)



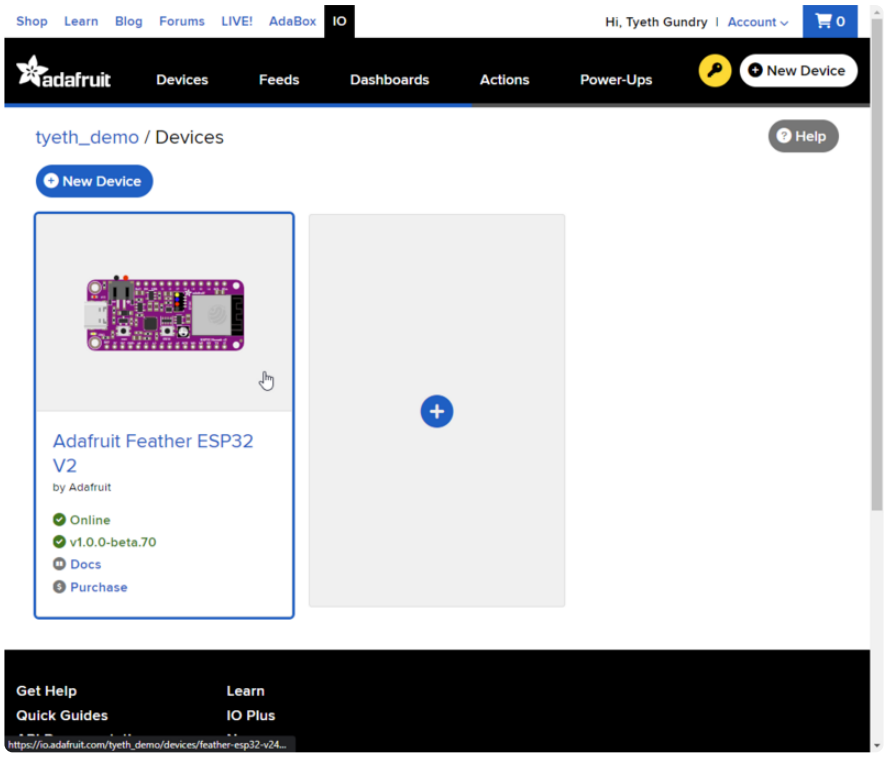


Board 3V to **sensor VIN (red wire on STEMMA QT)**
Board GND to **sensor GND (black wire on STEMMA QT)**
Board SCL to **sensor SCL (yellow wire on STEMMA QT)**
Board SDA to **sensor SDA (blue wire on STEMMA QT)**

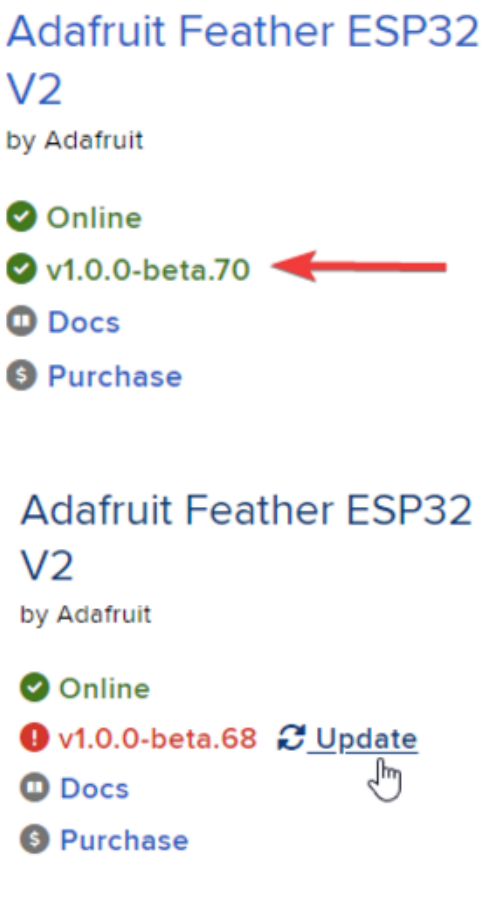
Usage

Connect your board to Adafruit IO Wippersnapper and navigate to the WipperSnapper board list **(https://adafru.it/TAu).**

On this page, **select the WipperSnapper board you're using** to be brought to the board's interface page.



If you do not see your board listed here - you need [to connect your board to Adafruit IO (https://adafru.it/Vfd)](#) first.

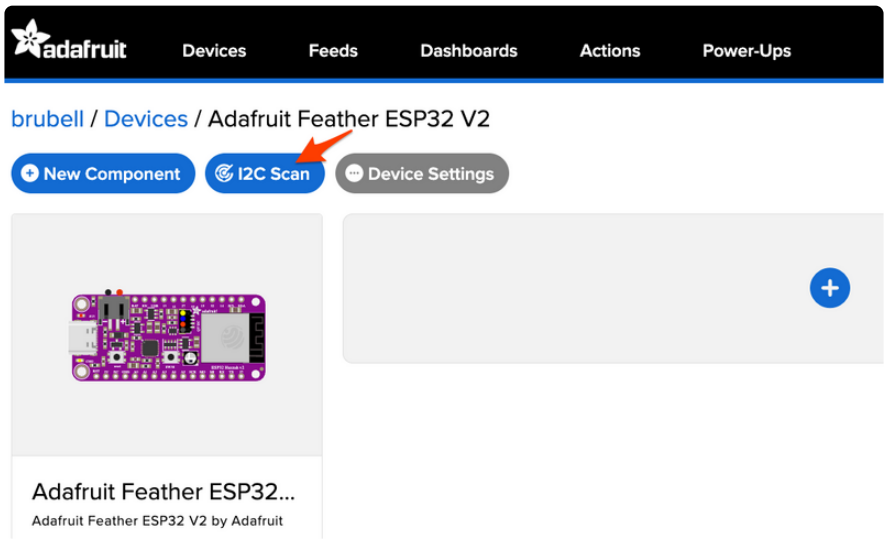


On the device page, quickly **check that you're running the latest version of the WipperSnapper firmware**.

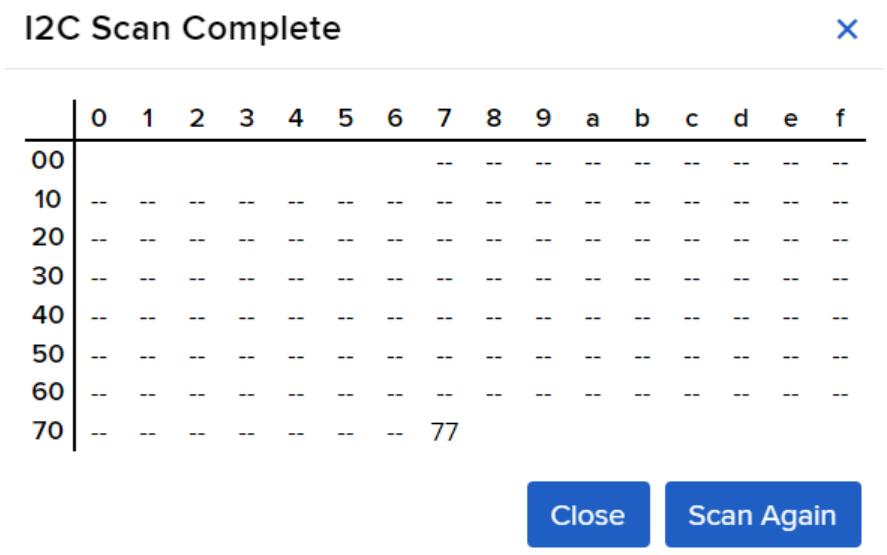
The device tile on the left indicates the version number of the firmware running on the connected board.

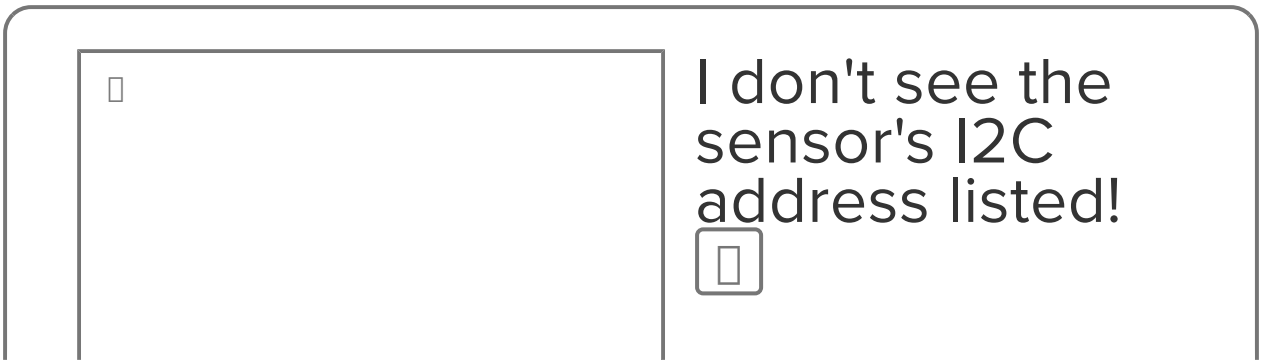
If the firmware version is green with a checkmark - continue with this guide.
If the firmware version is red with an exclamation mark "!" - [update to the latest WipperSnapper firmware (https://adafru.it/Vfd)](#) on your board before continuing.

Next, make sure the sensor is plugged into your board and click the **I2C Scan** button.



You should see the BMP388's default I2C address of `0x77` pop-up in the I2C scan list.





First, double-check the connection and/or wiring between the sensor and the board.

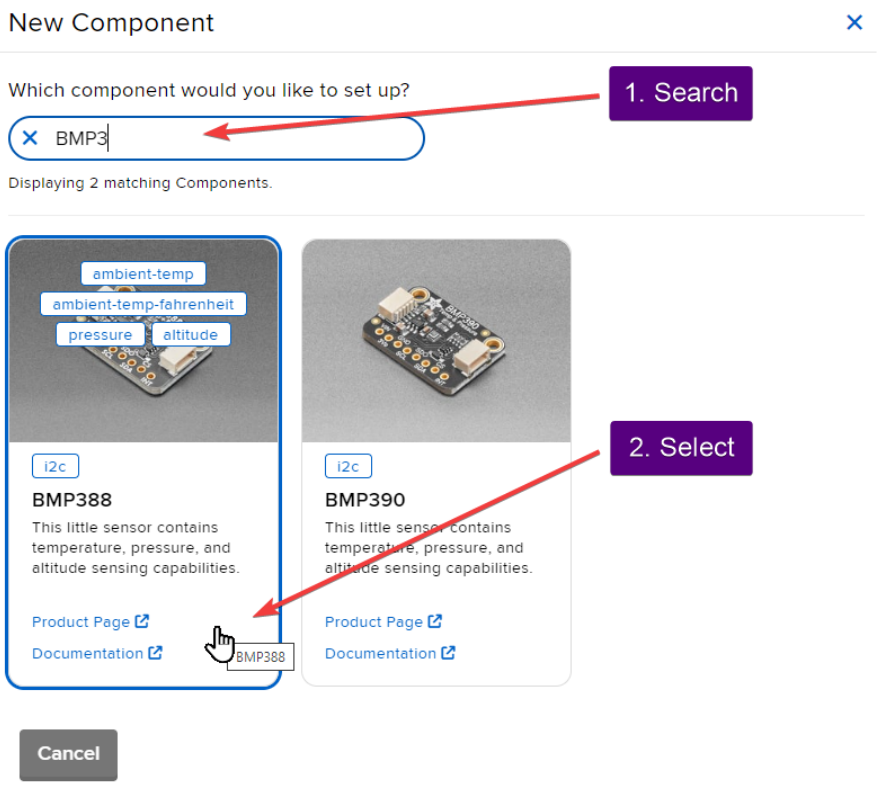
Then, reset the board and let it re-connect to Adafruit IO WipperSnapper.

With the sensor detected in an I2C scan, you're ready to add the sensor to your board.

Click the New Component button or the + button to bring up the component picker.



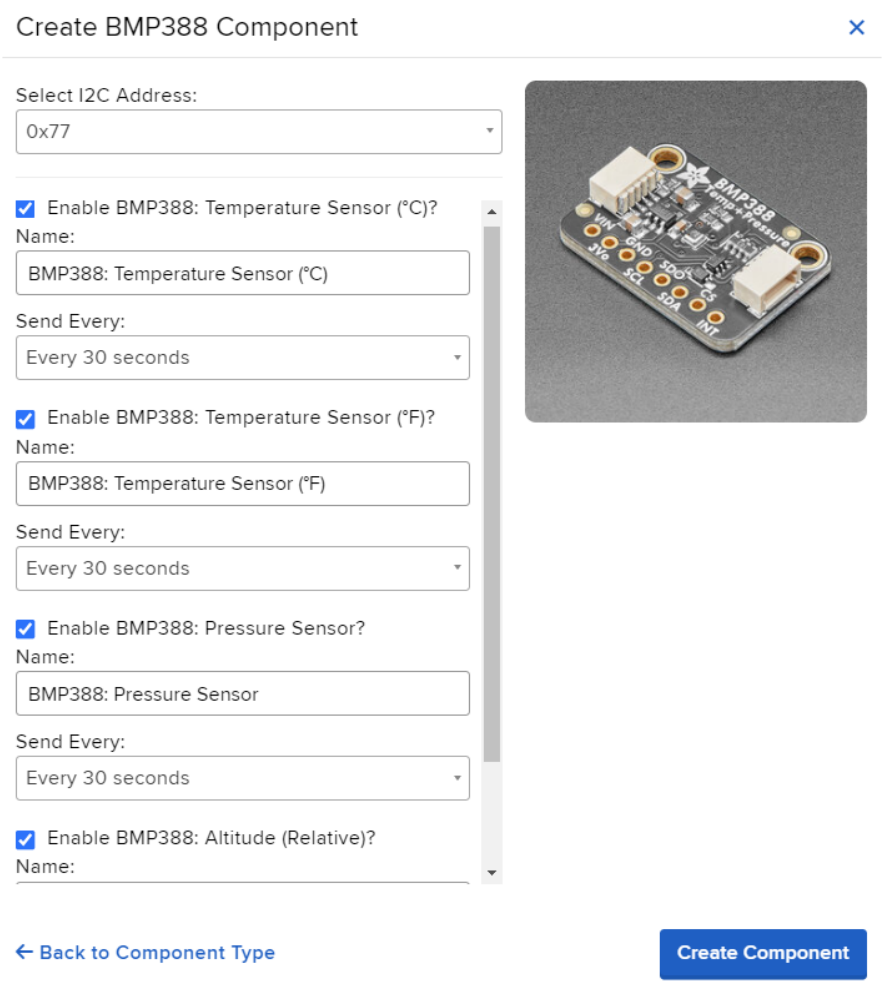
Adafruit IO supports a large amount of components. To quickly find your sensor, type `BMP3` into the search bar, then select the **BMP388** or **BMP390** component.



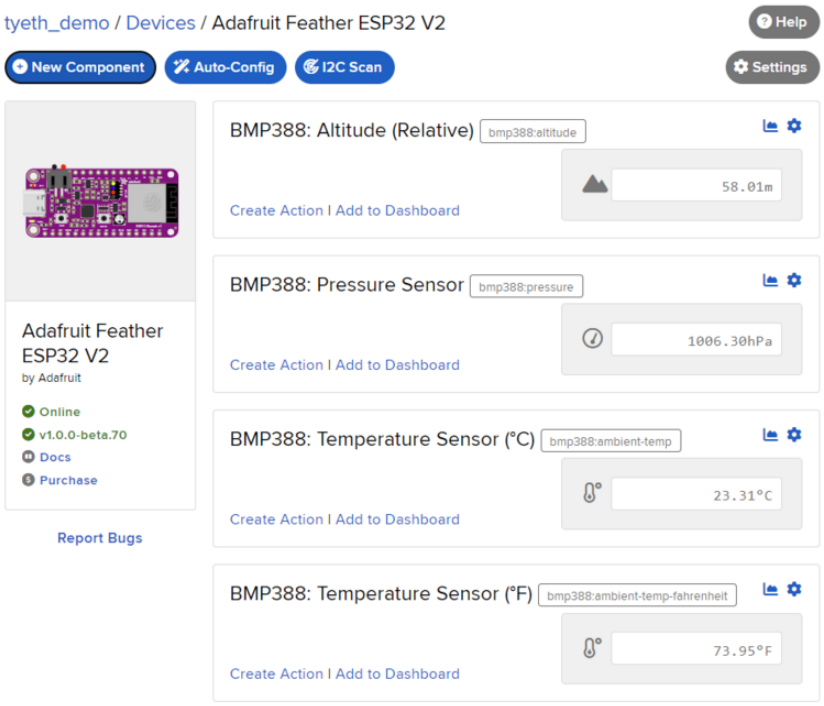
On the component configuration page, the BMP388's sensor address should be listed along with the sensor's settings.

The **Send Every** option is specific to each sensor's measurements. This option will tell the Feather how often it should read from the BMP388 sensor and send the data to Adafruit IO. Measurements can range from every 30 seconds to every 24 hours.

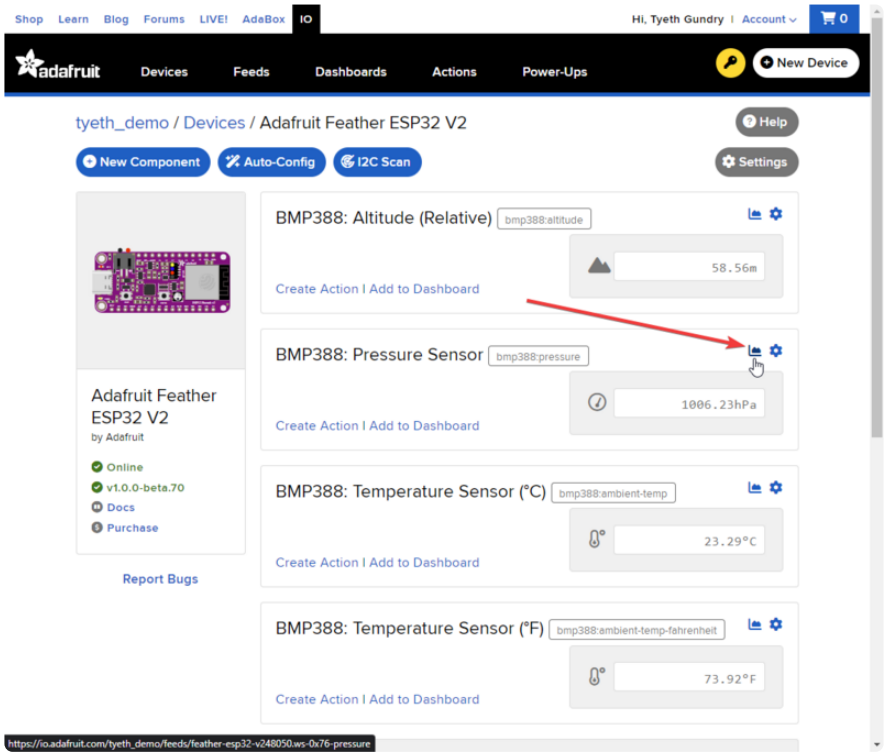
For this example, set the **Send Every** interval to every 30 seconds.



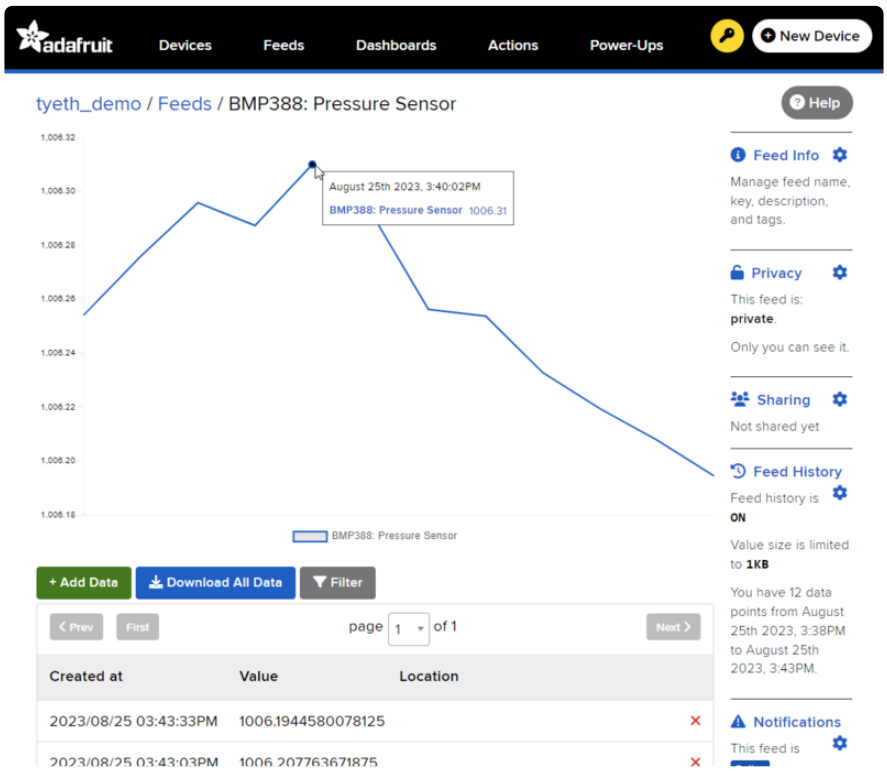
Your device interface should now show the sensor components you created. After the interval you configured elapses, WipperSnapper will automatically read values from the sensor(s) and send them to Adafruit IO.



To view the data that has been logged from the sensor, click on the graph next to the sensor name.



Here you can see the feed history and edit things about the feed such as the name, privacy, webhooks associated with the feed and more. If you want to learn more about how feeds work, check out this page (https://adafru.it/10aZ).

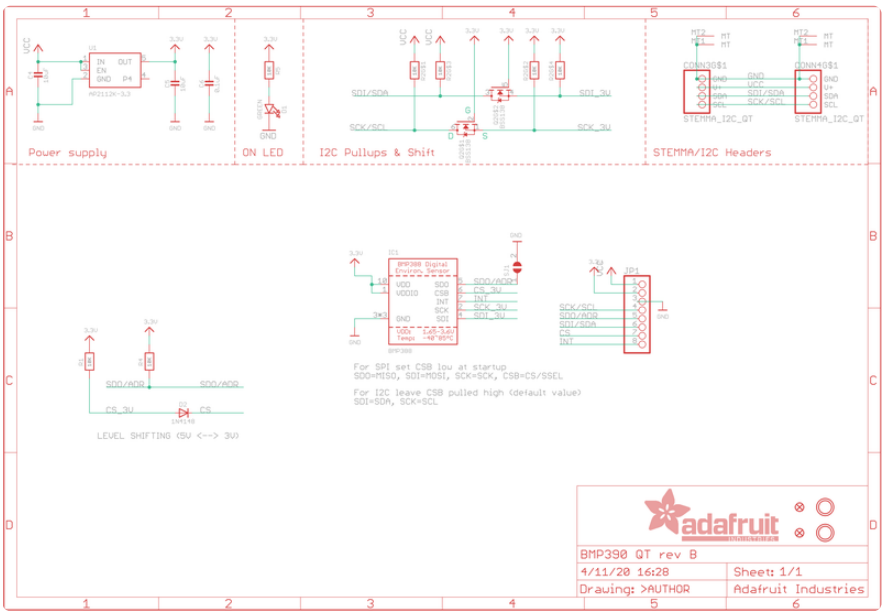


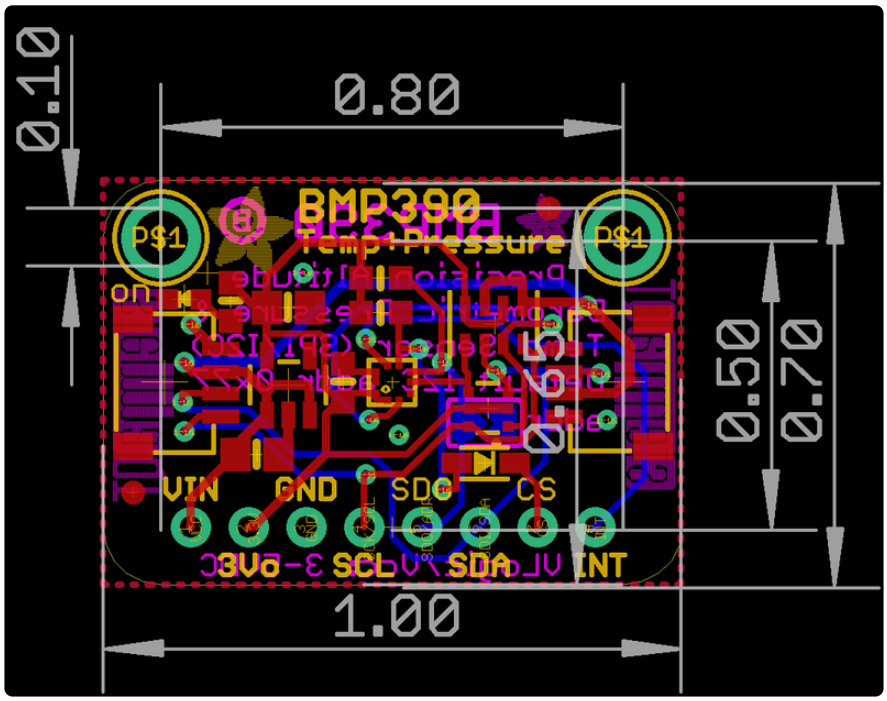
Downloads

Files

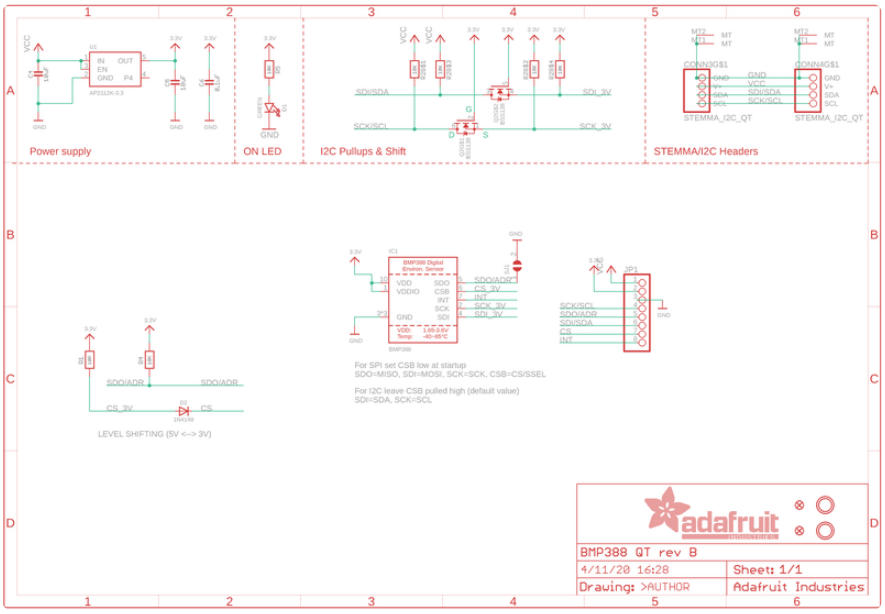
- BMP388 datasheet (https://adafru.it/19hf)
- BMP390 datasheet (https://adafru.it/Otb)
- BMP388 STEMMA QT Fritzing object in the Adafruit Fritzing Library (https://adafru.it/QEE)
- BMP388 Original Version Fritzing object in the Adafruit Fritzing Library (https://adafru.it/EfM)
- BMP390 Fritzing object in the Adafruit Fritzing Library (https://adafru.it/Otc)
- EagleCAD PCB files on GitHub (https://adafru.it/Otd)
- 3D models for BMP388 QT (https://adafru.it/19hA)
- 3D models for BMP390 QT (https://adafru.it/19hB)

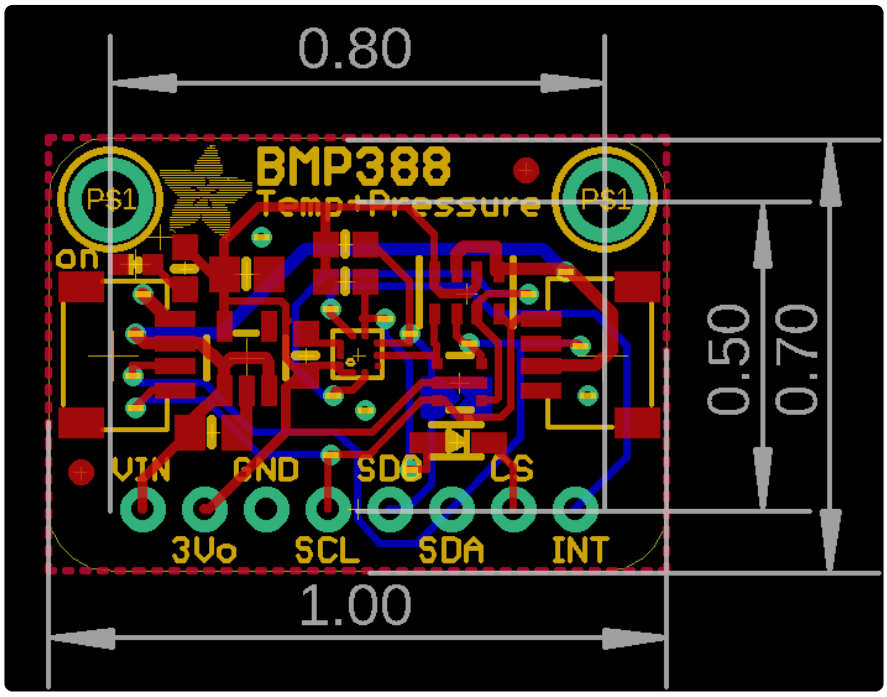
Schematic and Fab Print for BMP390



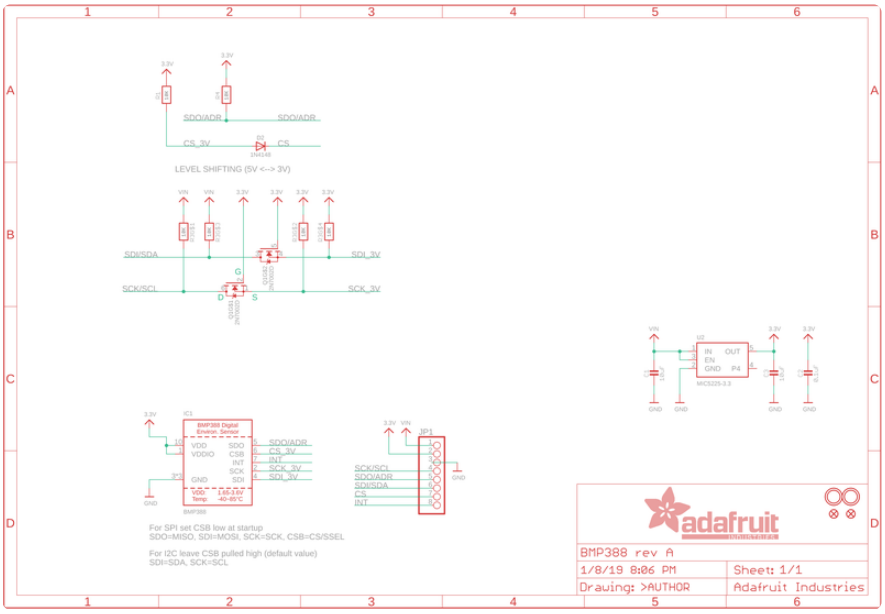


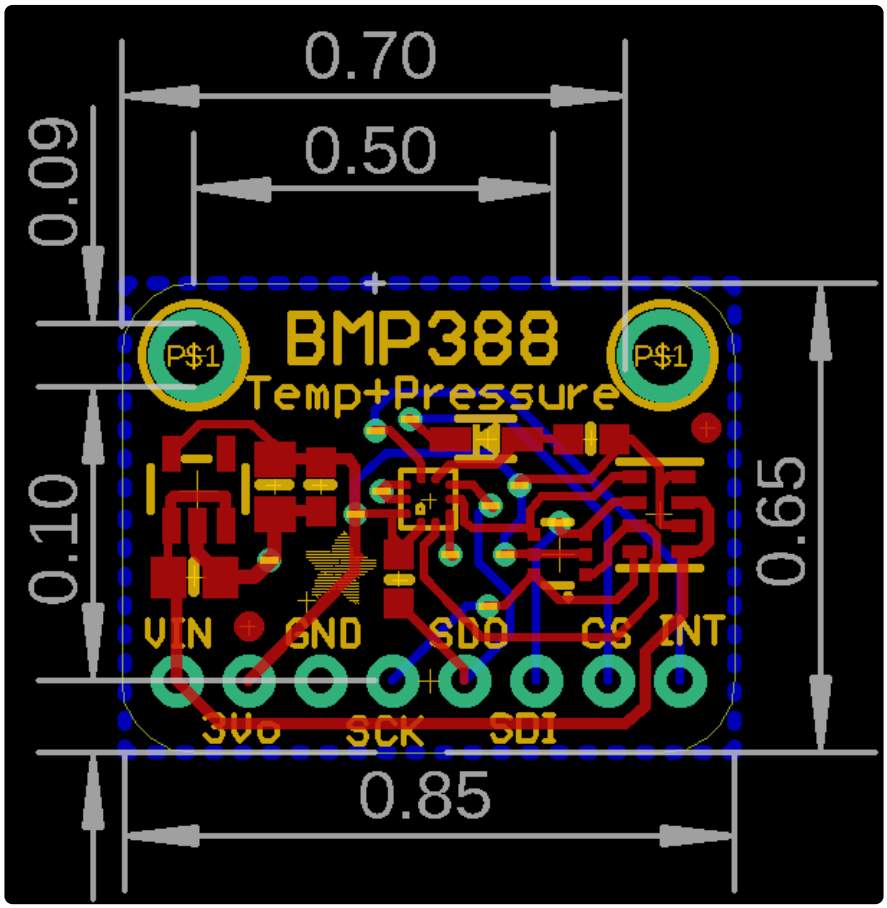
Schematic and Fab Print for BMP388 STEMMA QT Version



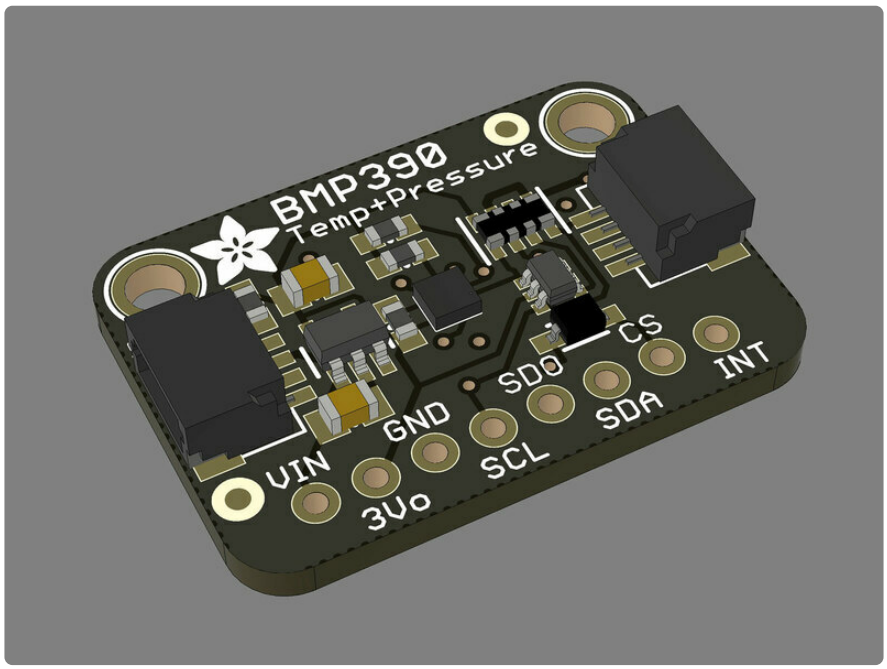


Schematic and Fab Print for BMP388 Original Version





3D Model of BMP390



3D Model of BMP388 QT

